

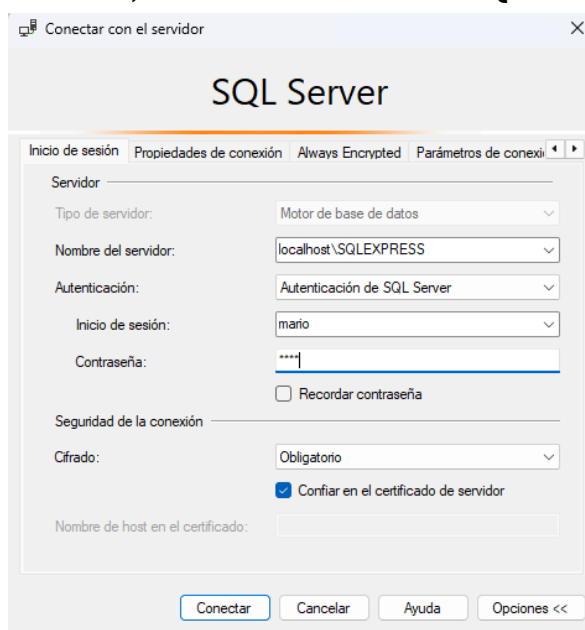
Guía paso a paso: de SQL Server (SSMS) a Azure SQL, ADF, Databricks, Lakehouse, Power BI y ML

1. Migrar datos de SQL Server (SSMS) a Azure SQL Database

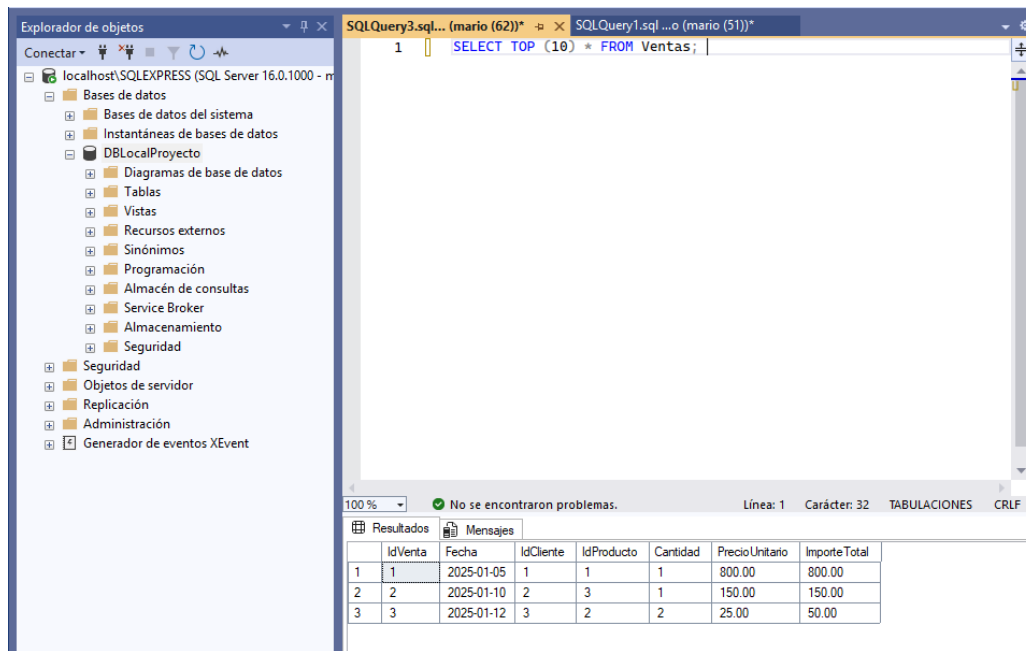
1.1. Crear y preparar la base de datos de origen en SQL Server local

En este paso se crea la base de datos de origen en el servidor SQL local y se insertan datos de ejemplo que luego se migrarán a Azure SQL Database. [learn.microsoft](https://learn.microsoft.com/es-es/sql/azure-sql-database/migrate-sql-server-to-azure-sql-database)

1. En SSMS, conectarse al **servidor SQL local** (origen).



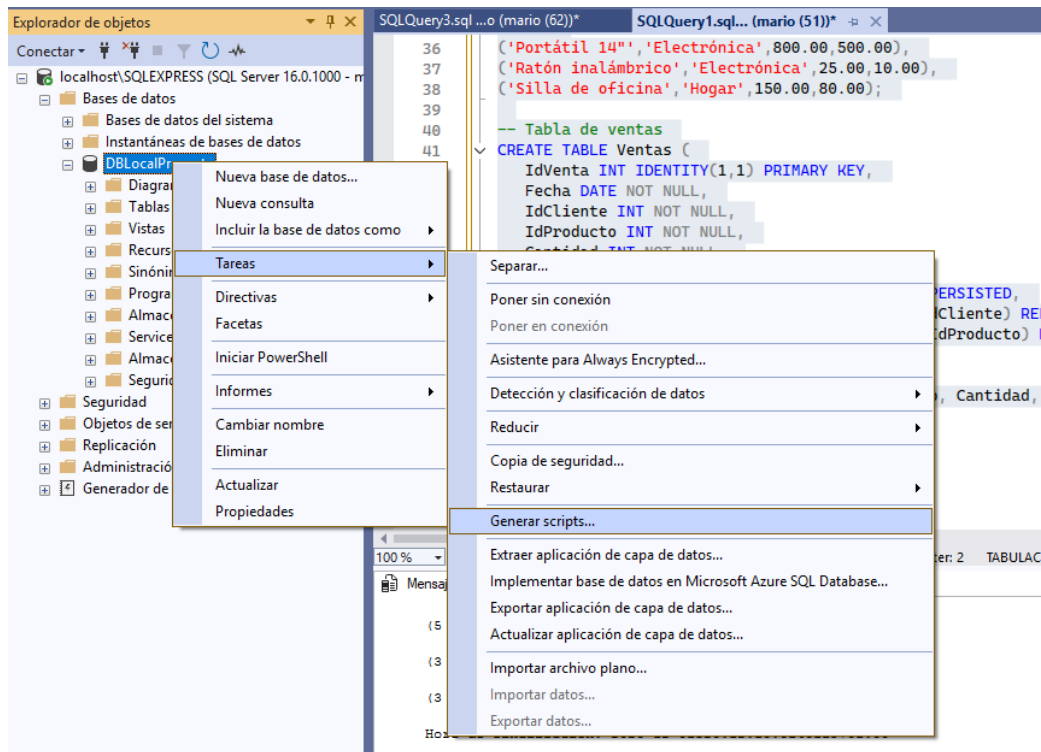
2. Crear la base de datos de origen ejecutando el siguiente script, antes pulsando click derecho en el servidor y en nueva consulta puedes copiar y pegar el script de creacion de base de datos (base_datos.sql) o abrirlo en ssms y ejecutarlo:
3. Verificar que hay datos ejecutando haciendo un show (top_ventas.sql):



1.2. Generar el script de migración desde la base local (Generate Scripts)

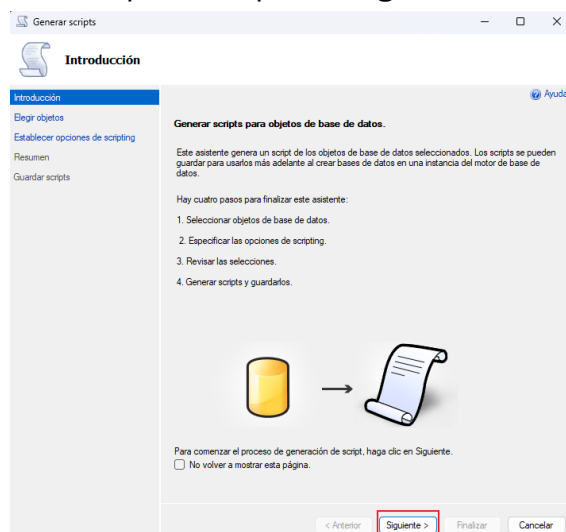
Ahora se va a generar un script T-SQL que contenga tanto la estructura de las tablas como los datos para poder recrearlos en Azure SQL Database.

1. En SSMS, seguir conectado al **servidor local** y localizar la base DBLocalProyecto en el Object Explorer.
2. Clic derecho sobre DBLocalProyecto → **Tasks** → **Generate Scripts....**



3. En el asistente de generación de scripts:

a. Primera pantalla: pulsar **Siguiente**.



b. Pantalla de selección: elegir **Seleccionar objetos de base de datos específicos**.

c. Marcar las tablas Clientes, Productos y Ventas (y cualquier otra que se quiera migrar).

Seleccione los objetos de la base de datos para crear el script.

☐ Crear un script a partir de toda la base de datos y todos los objetos de esta

☒ Seleccionar objetos de base de datos específicos

☒ Tablas

- ☒ dbo.Clientes
- ☒ dbo.Productos
- ☒ dbo.Ventas

d. Pulsar **Siguiente**.

4. En **Establecer opciones de scripting**:

a. Elegir **Abrir en la ventana Nueva consulta**.

Generar scripts

Establecer opciones de scripting

Introducción
Elegir objetos
Establecer opciones de scripting
Resumen
Guardar scripts

Especifique cómo se deben guardar los scripts

☐ Guardar como cuaderno Avanzadas

☐ Guardar como archivo de script

 Archivos que se van a generar:

- ☒ Archivo de script único
- ☐ Un archivo de script por objeto

Nombre de archivo: C:\Users\mario\Documents\script.sql ...

☒ Sobrescribir el archivo existente

Guardar como:

- ☒ Texto Unicode
- ☐ Texto ANSI

☐ Guardar en el portapapeles

☒ **Abrir en la ventana Nueva consulta**

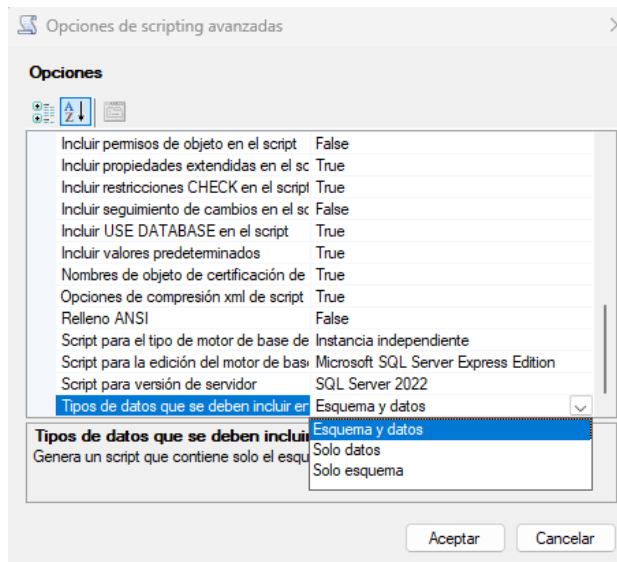
< Anterior Siguiente > Finalizar Cancelar

b. Hacer clic en **Avanzadas**....

Avanzadas

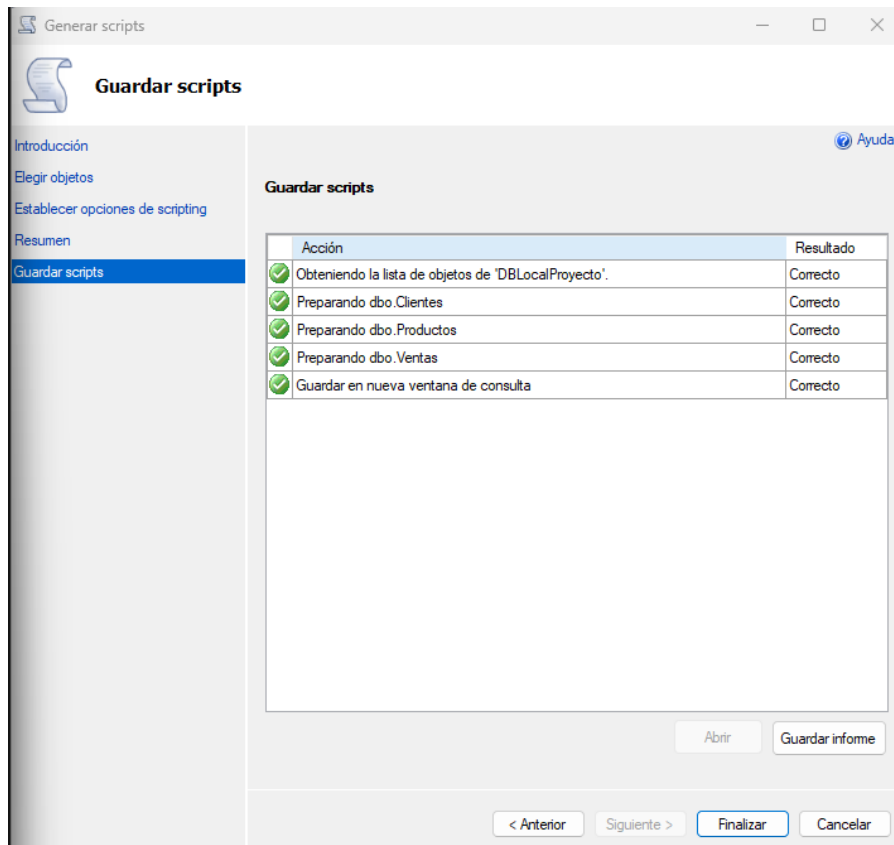
5. En las opciones avanzadas:

a. Buscar **Tipos de datos que se van a incluir en el script** y seleccionar **Esquema y datos** (para incluir estructura y filas).



b. Pulsar **Aceptar**.

6. Volver al asistente, pulsar **Next** y, finalmente, **Finish**.



7. Revisar que se ha generado un script T-SQL con:
- Instrucciones CREATE TABLE para las tablas seleccionadas.
 - Instrucciones INSERT INTO con los datos.

```

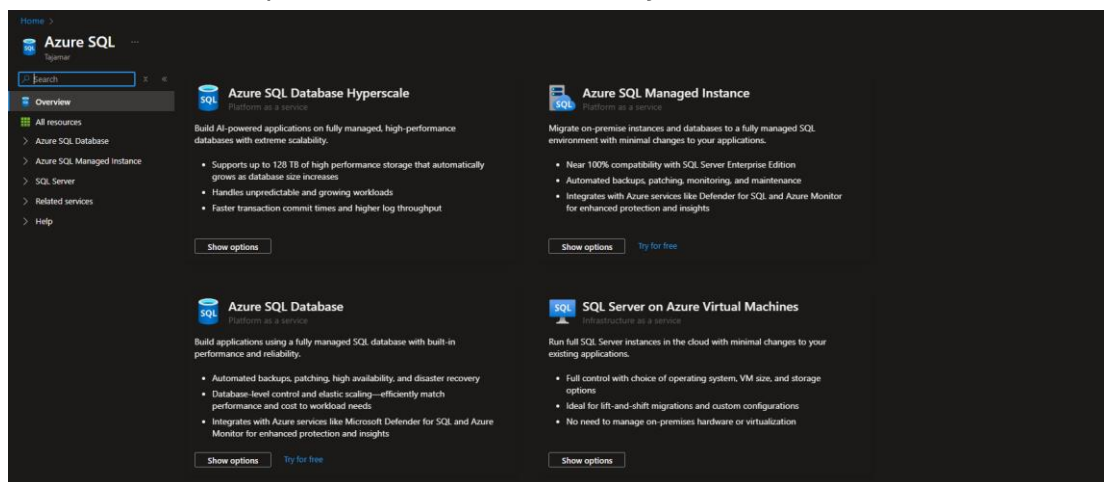
SQLQuery4.sql - r (marco CSS)
1 USE [DBLocalProyecto]
2 GO
3 /***** Object: Table [dbo].[Clientes]    Script Date: 01/12/2025 20:26:19 *****/
4 SET ANSI_NULLS ON
5 GO
6 SET QUOTED_IDENTIFIER ON
7 GO
8 CREATE TABLE [dbo].[Clientes](
9     [IdCliente] [int] IDENTITY(1,1) NOT NULL,
10    [Nombre] [nvarchar](100) NOT NULL,
11    [Segmento] [nvarchar](50) NOT NULL,
12    [Ciudad] [nvarchar](100) NOT NULL,
13    [Pais] [nvarchar](100) NOT NULL,
14    [FechaAlta] [date] NOT NULL,
15    PRIMARY KEY CLUSTERED
16    (
17        [IdCliente] ASC
18    ) WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENTIAL_KEY = OFF) ON [PRIMARY]
19 ) ON [PRIMARY]
20 GO
21 /***** Object: Table [dbo].[Productos]    Script Date: 01/12/2025 20:26:19 *****/
22 SET ANSI_NULLS ON
23 GO
24 SET QUOTED_IDENTIFIER ON
25 GO
26 CREATE TABLE [dbo].[Productos](
27     [IdProducto] [int] IDENTITY(1,1) NOT NULL,
28    [NombreProducto] [nvarchar](100) NOT NULL,
29    [Categoria] [nvarchar](50) NOT NULL,
30    [PrecioBase] [decimal](10, 2) NOT NULL,
31    [Coste] [decimal](10, 2) NOT NULL,
32    PRIMARY KEY CLUSTERED
33    (
34        [IdProducto] ASC
35    ) WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENTIAL_KEY = OFF) ON [PRIMARY]
36 ) ON [PRIMARY]
37 GO
38 /***** Object: Table [dbo].[Ventas]    Script Date: 01/12/2025 20:26:19 *****/
39 SET ANSI_NULLS ON
40 GO
41 SET QUOTED_IDENTIFIER ON
42 GO
43 CREATE TABLE [dbo].[Ventas](
44     [IdVenta] [int] IDENTITY(1,1) NOT NULL,
45    [Fecha] [date] NOT NULL,
46    [IdCliente] [int] NOT NULL,
47    [IdProducto] [int] NOT NULL,
48    [Cantidad] [int] NOT NULL,
49    [PrecioUnitario] [decimal](10, 2) NOT NULL,
50    [Total] [decimal](10, 2) NOT NULL,
51 ) ON [PRIMARY]

```

8. Copiar todo el script generado (o dejar abierta la ventana de query donde se ha creado).

1.3. Crear Azure SQL Database

1. Abrir el portal de Azure (<https://portal.azure.com>).
2. En la barra de búsqueda, escribir “Azure SQL” y hacer clic en “Azure SQL”.



3. Pulsar en “+ Crear” y elegir la opción “Base de datos de Azure SQL Database”.
4. Rellenar los datos:
 - a. Suscripción: seleccionar la suscripción disponible.
 - b. Grupo de recursos: crear uno nuevo o usar uno existente (por ejemplo, rg-proyecto-clase).
 - c. Nombre de la base de datos: por ejemplo, DBProyectoClase.
 - d. Servidor: hacer clic en “Crear nuevo” y definir:
 - i. Nombre del servidor, por ejemplo, srv-proyecto-clase.
 - ii. Región (la misma que el resto de recursos que se van a crear).
 - iii. Método de autenticación: SQL.
 - iv. Usuario administrador y contraseña (anotarlos).

Microsoft Azure | Search resources, services, and docs (0+)

Home > Azure SQL > Create SQL Database >

Create SQL Database Server

Microsoft

Server details

Enter required settings for this server, including providing a name and location. This server will be created in the same subscription and resource group as your database.

Server name * ✓
 Location * ✓

Authentication

Azure Active Directory (Azure AD) is now Microsoft Entra ID. [Learn more](#)

Select your preferred authentication method for accessing this server. Create a server admin login and password to access your server with SQL authentication. Select only Microsoft Entra authentication [learn more](#) or using an existing Microsoft Entra user, group, or application as Microsoft Entra admin [learn more](#) or, or select both SQL and Microsoft Entra authentication.

Authentication method
☐ Use Microsoft Entra-only authentication
☐ Use both SQL and Microsoft Entra authentication
☒ Use SQL authentication

Server admin login * ✓
 Password * ✓
 Confirm password * ✓

[Feedback](#)

[OK](#)

- e. Elastic pool seleccionamos que no
- f. Workload environment: development
- g. Compute + storage el que está por defecto: General Purpose - Serverless
- h. Backup storage redundancy: Locally-redundant backup storage

Home > Azure SQL >

Create SQL Database

Microsoft

Select the subscription to manage deployed resources and costs. Use resource groups that already exist to organize and manage all your resources.

Subscription * ✓
 Resource group * ✓
[Create new](#)

Database details

Enter required settings for this database, including picking a logical server and configuring the compute and storage resources

Database name * ✓
 Server * ✓
[Create new](#)

Want to use SQL elastic pool? ☐ Yes ☒ No

Workload environment
☒ Development
☐ Production

Default settings provided for Development workloads. Configurations can be modified as needed.

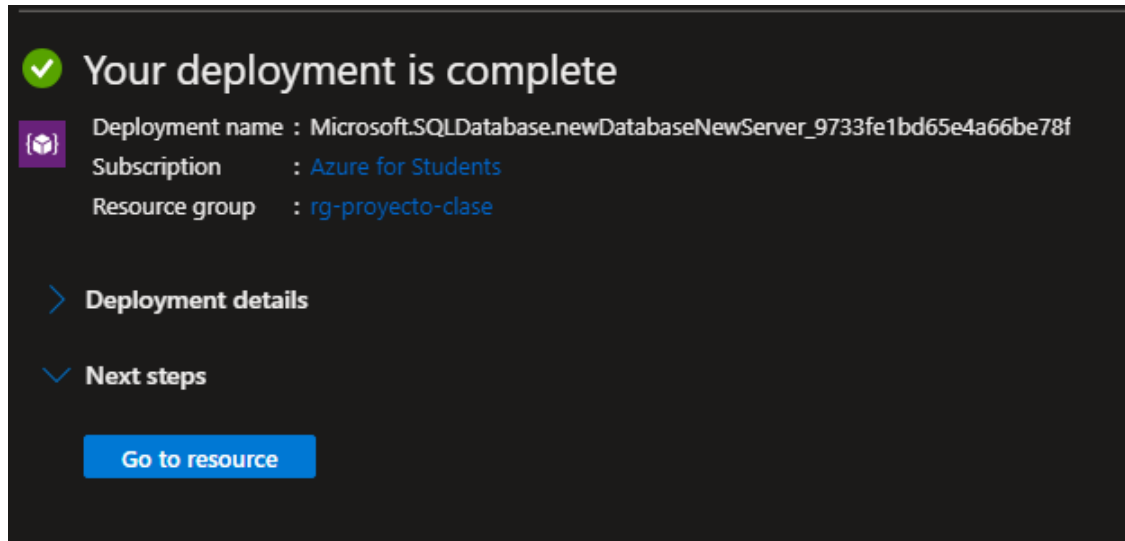
Compute + storage * ✓
 Standard-series (Gen5), 1 vCore, 32 GB storage, zone redundant disabled
[Configure database](#)

Backup storage redundancy

Choose how your PITR and LTR backups are replicated. Geo restore or ability to recover from regional outage is only available when geo-redundant storage is selected.

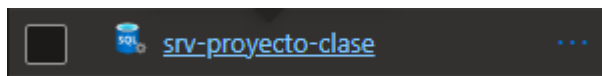
- Backup storage redundancy ☐ ☒ Locally-redundant backup storage
☐ Zone-redundant backup storage

5. Hacer clic en “Revisar y crear” y luego en “Crear”.
6. Esperar a que termine el despliegue.

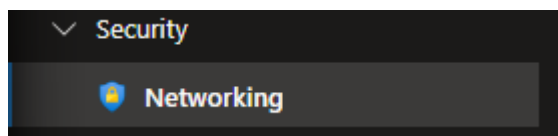


1.5. Configurar firewall y redes de Azure SQL

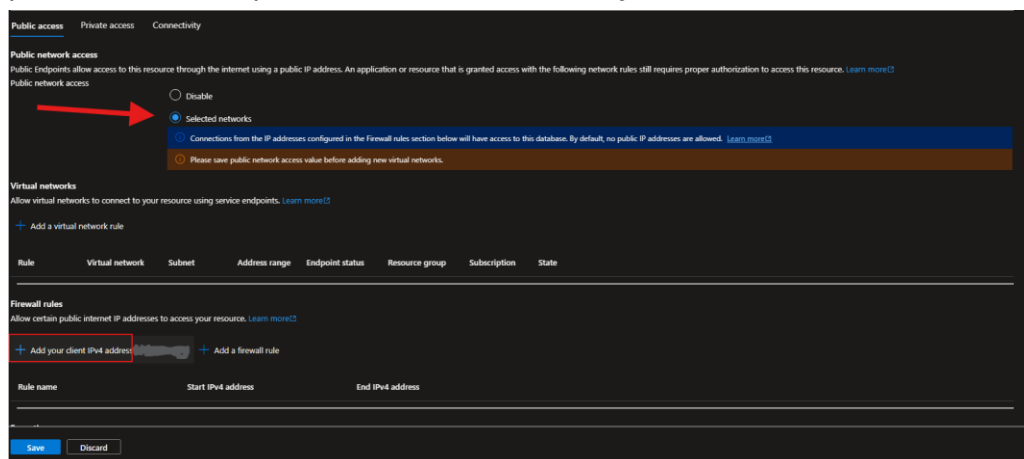
1. En el portal de Azure, ir al grupo de recursos creado “rg-proyecto-clase”
2. Entrar en el servidor que hemos creado antes (srv-proyecto-clase.)



3. En el menú lateral izquierdo vamos a security y networking



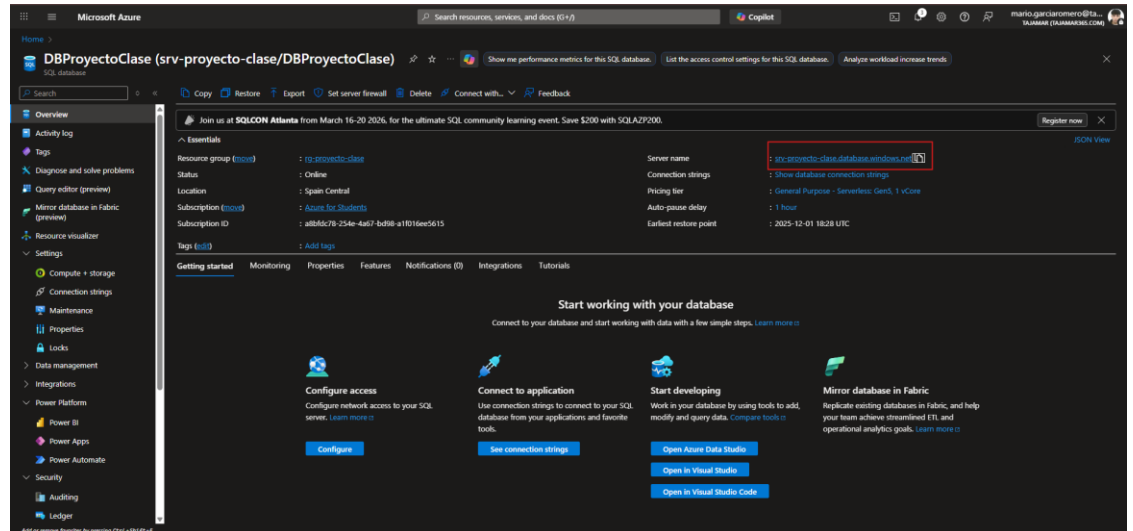
4. En el apartado public acces, seleccionamos en public network acces: *selected networks*, y más abajo seleccionamos *add your IPv4 addres* para registrar la IP pública desde la que se va a conectar SSMS y una vez hecho le damos a save.



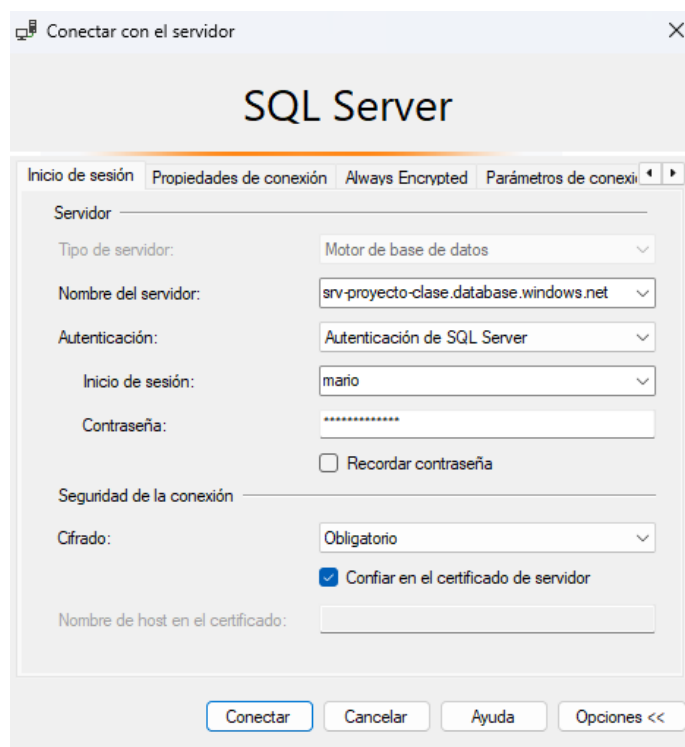
5. Guardar los cambios y esperar a que se apliquen.

1.6. Conectarse a Azure SQL desde SSMS

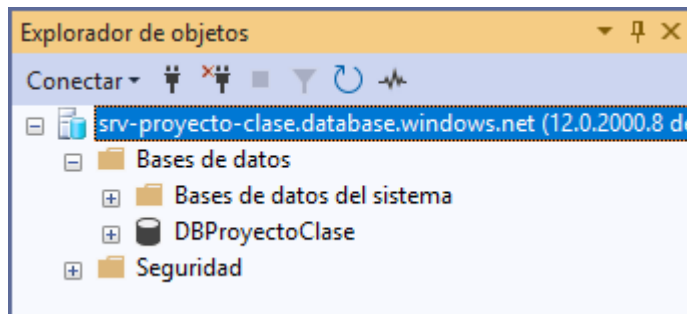
1. Abrir SQL Server Management Studio (SSMS) en el equipo local.
2. En la ventana “Connect to Server”:
 - a. Server type: Database Engine.
 - b. Server name: el nombre FQDN del servidor SQL de Azure, por ejemplo, srv-proyecto-clase.database.windows.net.



- c. Authentication: “SQL Server Authentication”.
- d. Login: el usuario administrador configurado al crear el servidor.
- e. Password: la contraseña asociada.



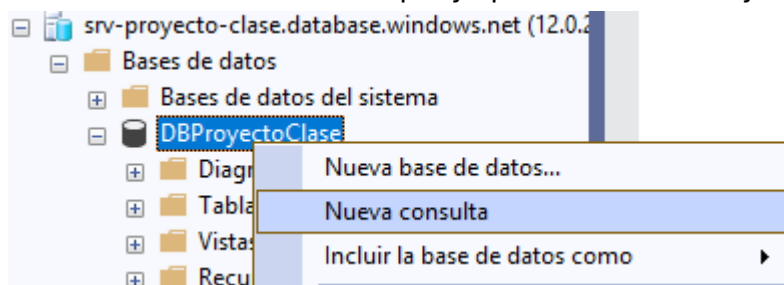
3. Pulsar en “Connect”. Si todo está bien configurado, aparecerá el servidor con la base de datos DBProyectoClase en el Object Explorer.



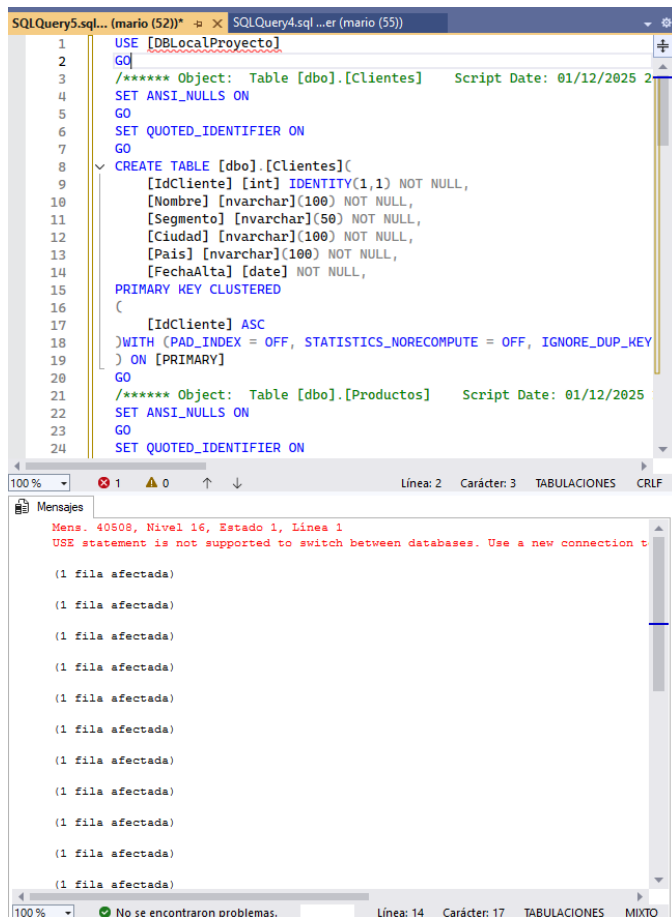
1.6. Ejecutar el script en la base de datos de Azure SQL

En este paso se recrean tablas y datos en la base de destino DBProyectoClase de Azure SQL Database usando el script generado

1. En SSMS, en el **servidor de Azure SQL** (srv-proyecto-clase.database.windows.net) nos metemos en el object explorer.
2. En Object Explorer, seleccionar la base de datos **DBProyectoClase** (destino).
3. Abrir una nueva ventana de query apuntando a DBProyectoClase.



4. Pegar el script T-SQL generado en el paso 1.5.
5. Ejecutar el script completo (botón **Execute** o tecla F5) y esperar a que finalice.



```
1 USE [DBLocalProyecto]
2 GO
3 /***** Object: Table [dbo].[Clientes]    Script Date: 01/12/2025 2
4 SET ANSI_NULLS ON
5 GO
6 SET QUOTED_IDENTIFIER ON
7 GO
8 CREATE TABLE [dbo].[Clientes](
9     [IdCliente] [int] IDENTITY(1,1) NOT NULL,
10     [Nombre] [nvarchar](100) NOT NULL,
11     [Segmento] [nvarchar](50) NOT NULL,
12     [Ciudad] [nvarchar](100) NOT NULL,
13     [Pais] [nvarchar](100) NOT NULL,
14     [FechaAlta] [date] NOT NULL,
15     PRIMARY KEY CLUSTERED
16     (
17         [IdCliente] ASC
18     )WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY
19     ) ON [PRIMARY]
20 GO
21 /***** Object: Table [dbo].[Productos]    Script Date: 01/12/2025
22 SET ANSI_NULLS ON
23 GO
24 SET QUOTED_IDENTIFIER ON
```

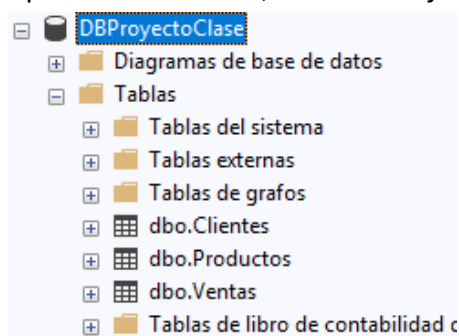
Mens. 40508, Nivel 16, Estado 1, Línea 1
USE statement is not supported to switch between databases. Use a new connection to the target database.

(1 fila afectada)
(1 fila afectada)
(1 fila afectada)
(1 fila afectada)
(1 fila afectada)
(1 fila afectada)
(1 fila afectada)
(1 fila afectada)
(1 fila afectada)
(1 fila afectada)

No se encontraron problemas.

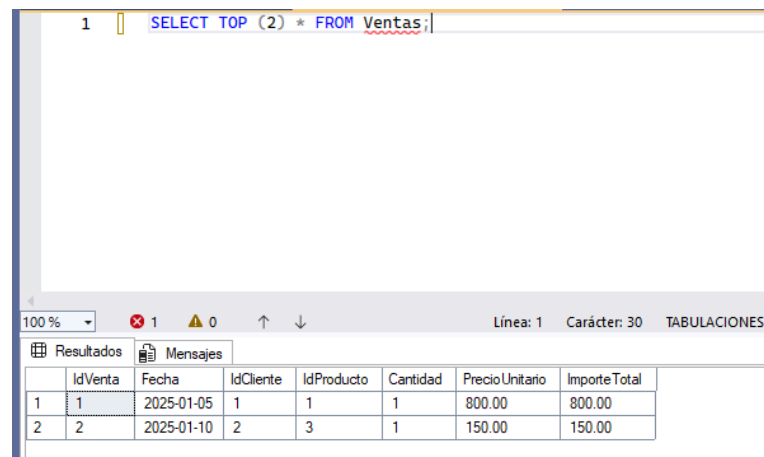
Da error porque el script generado incluye un USE DBLocalProyecto; al principio, y en Azure SQL Database no se permite usar USE para cambiar de base pero no interfiere en la inserción de datos.

6. Verificar que las tablas se han creado correctamente:
 - a. En Object Explorer, expandir DBProyectoClase → **Tables** y comprobar que aparecen Clientes, Productos y Ventas.



7. Verificar que los datos se han copiado correctamente ejecutando, por ejemplo:

SELECT TOP (2) * FROM Ventas;



The screenshot shows a SQL query editor with the query `SELECT TOP (2) * FROM Ventas;` entered. Below the editor, the results are displayed in a table with the following columns: IdVenta, Fecha, IdCliente, IdProducto, Cantidad, PrecioUnitario, and ImporteTotal. The results table contains two rows of data.

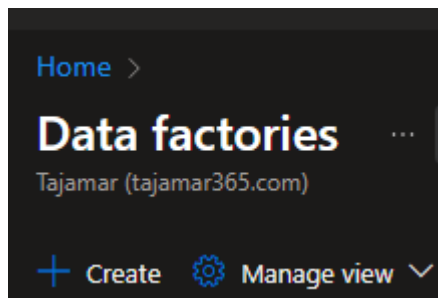
	IdVenta	Fecha	IdCliente	IdProducto	Cantidad	PrecioUnitario	ImporteTotal
1	1	2025-01-05	1	1	1	800.00	800.00
2	2	2025-01-10	2	3	1	150.00	150.00

y comprobando que las filas coinciden con las de DBLocalProyecto en el servidor local.

2. Orquestación con Azure Data Factory (ADF): de Azure SQL a Databricks / Delta Lake

2.1. Crear una instancia de Azure Data Factory

1. En el portal de Azure, buscar “Data Factory”.
2. Hacer clic en “+ Crear”.



3. Rellenar los datos:
 - a. Suscripción: la misma que los otros recursos.
 - b. Grupo de recursos: usar rg-proyecto-clase.
 - c. Nombre de la factoría de datos: por ejemplo, adf-proyecto-clase.
 - d. Región: la misma o cercana a la de la base de datos SQL.

Subscription * ⓘ Azure for Students

Resource group * ⓘ rg-proyecto-clase
[Create new](#)

Instance details

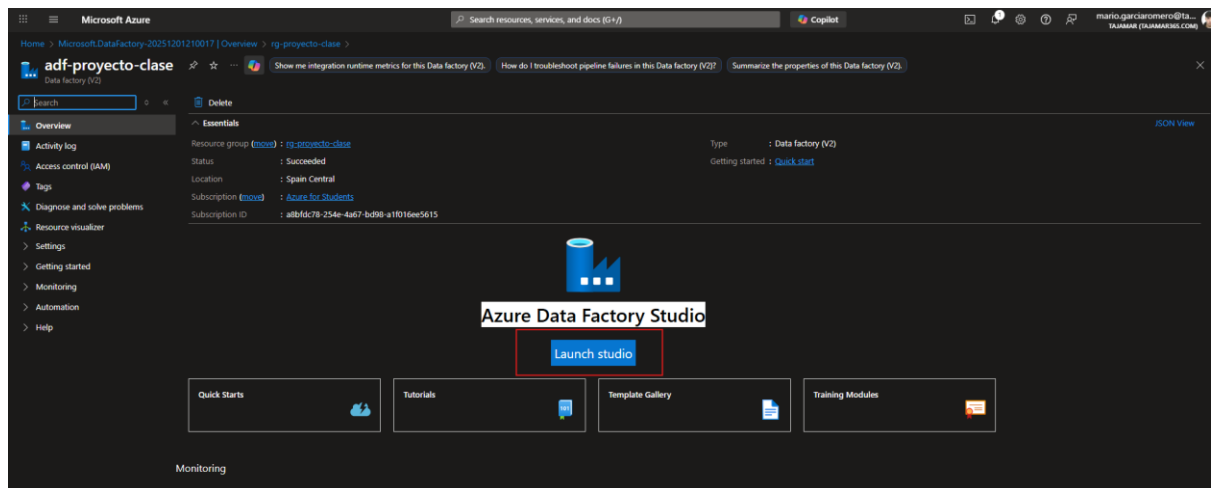
Name * ⓘ adf-proyecto-clase ✓

Region * ⓘ Spain Central

Version * ⓘ V2

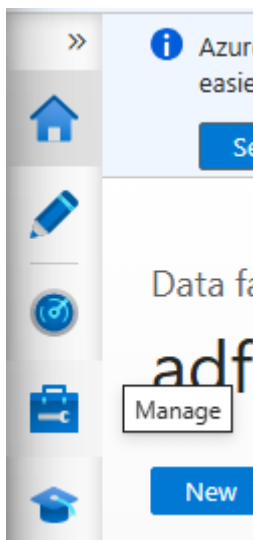
i We recommend you **upgrade to Fabric Data Factory** for a modern, unified data integration experience. [Try Fabric instead!](#)

4. Pulsar “Revisar y crear” y luego “Crear”.
5. Tras el despliegue, entrar al recurso y hacer clic en “Launch Studio” para abrir el entorno de ADF.

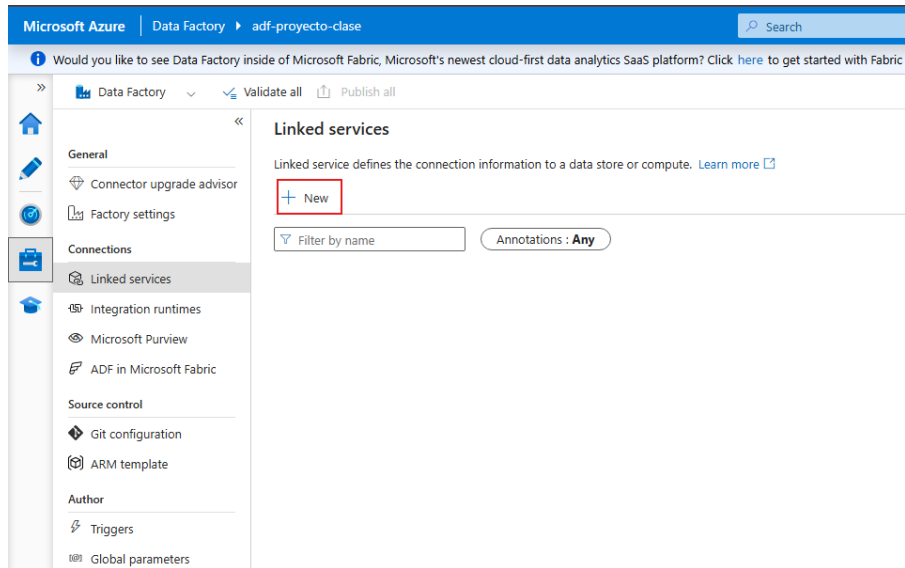


2.2. Crear Linked Service de origen (Azure SQL)

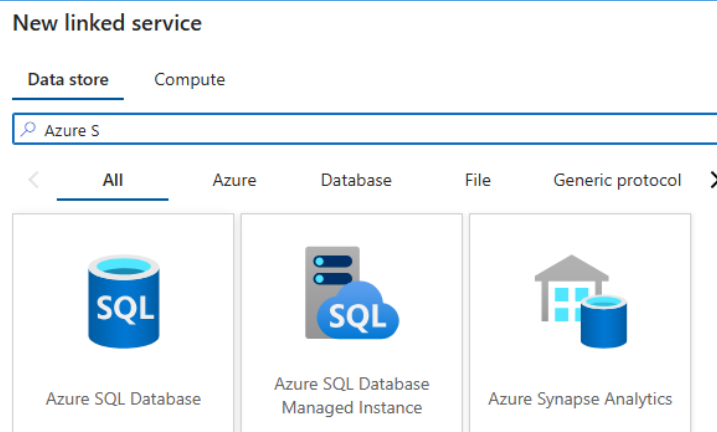
1. En Azure Data Factory Studio, ir al menú izquierdo y hacer clic en el icono de “Manage”.



2. Seleccionar “Linked services”.
3. Pulsar en “+ New”.



4. En la lista de conectores, seleccionar “Azure SQL Database”.



5. Configurar:
 - a. Nombre del Linked Service: LS_AzureSQL_Origen.
 - b. Server name: seleccionar el servidor srv-proyecto-clase.database.windows.net.
 - c. Database name: DBProyectoClase.
 - d. Authentication type: “SQL authentication”.
 - e. Username y password: los mismos que en la conexión desde SSMS.

New linked service
 Azure SQL Database [Learn more](#)

Name *
 LS_AzureSQL_Origen

Description

Connect via integration runtime *
 AutoResolveIntegrationRuntime

Version
☒ 2.0 (Recommended) ☐ 1.0
[Import from connection string](#)

Account selection method
☒ From Azure subscription ☐ Enter manually

Azure subscription
 Azure for Students (a8bfdc78-254e-4a67-bd98-a1f016ee5615)

Server name *
 srv-proyecto-clase

Database name *
 DBProyectoClase

Authentication type *
 SQL authentication

User name *
 mario

Password *
 Password

6. Hacer clic en “Test connection” para comprobar conectividad.
 Si al probar la conexión aparece un error similar a:

“Client with IP address ‘X.X.X.X’ is not allowed to access the server. To enable access, use the Azure Management Portal...”

Error details

Error code [SqlFailedToConnect](#) [Open troubleshooting guide in a new tab](#)

Details
 Cannot connect to SQL Database. Please contact SQL server team for further support. Server: 'srv-proyecto-clase.database.windows.net', Database: 'DBProyectoClase', User: 'mario'. Check the linked service configuration is correct, and make sure the SQL Database firewall allows the integration runtime to access. Cannot open server 'srv-proyecto-clase' requested by the login. Client with IP address '68.221.104.156' is not allowed to access the server. To enable access, use the Azure Management Portal or run `sp_set_firewall_rule` on the master database to create a firewall rule for this IP address or address range. It may take up to five minutes for this change to take effect.

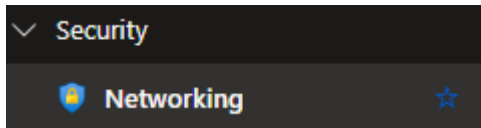
Activity ID 1ee33988-0de2-4338-b5ef-83e055980a84

Connection failed [More](#)

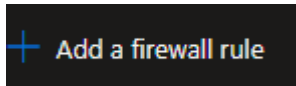
[Test connection](#) [Cancel](#)

significa que el firewall de Azure SQL no permite la IP que usa Azure Data Factory (integration runtime). En ese caso:

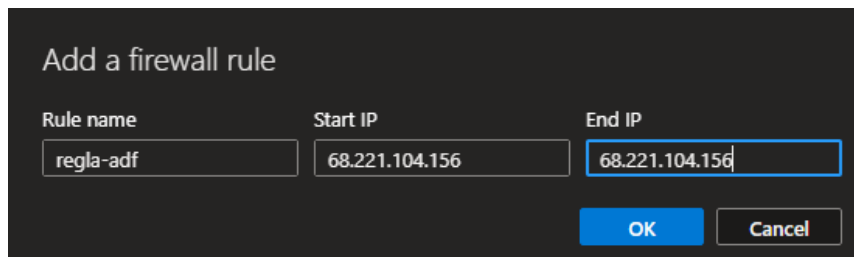
1. Ir al portal de Azure y abrir el servidor lógico srv-proyecto-clase
2. En el menú izquierdo, ir a Security > Networking.



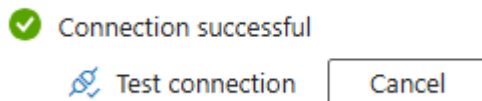
3. En la pestaña Public access, en la sección de Firewall rules, crear una nueva regla:



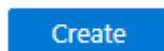
- 3.1. Nombre de la regla: por ejemplo, regla-adf.
- 3.2. Start IP y End IP: escribir la IP que aparece en el mensaje de error (por ejemplo, 68.221.104.156).

A screenshot of the 'Add a firewall rule' dialog box. It contains three input fields: 'Rule name' with the value 'regla-adf', 'Start IP' with the value '68.221.104.156', and 'End IP' with the value '68.221.104.156'. At the bottom right are 'OK' and 'Cancel' buttons.

4. Guardar los cambios y esperar a que se apliquen.
5. Volver a Azure Data Factory, abrir el Linked Service LS_AzureSQL_Origen y pulsar otra vez Test connection, que ahora debería funcionar correctamente.

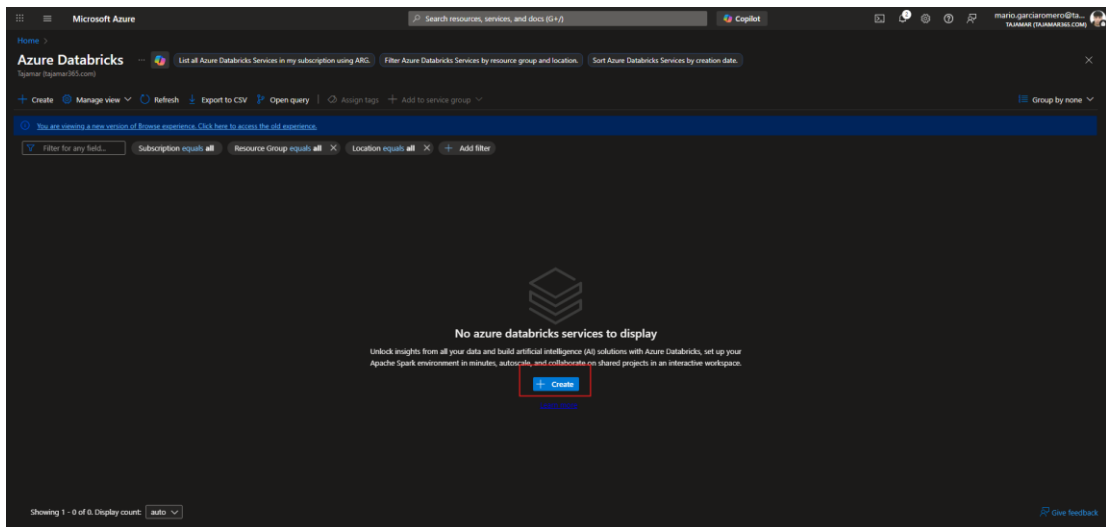


6. Finalmente, pulsar "Create".



2.3. Crear workspace y clúster de Azure Databricks

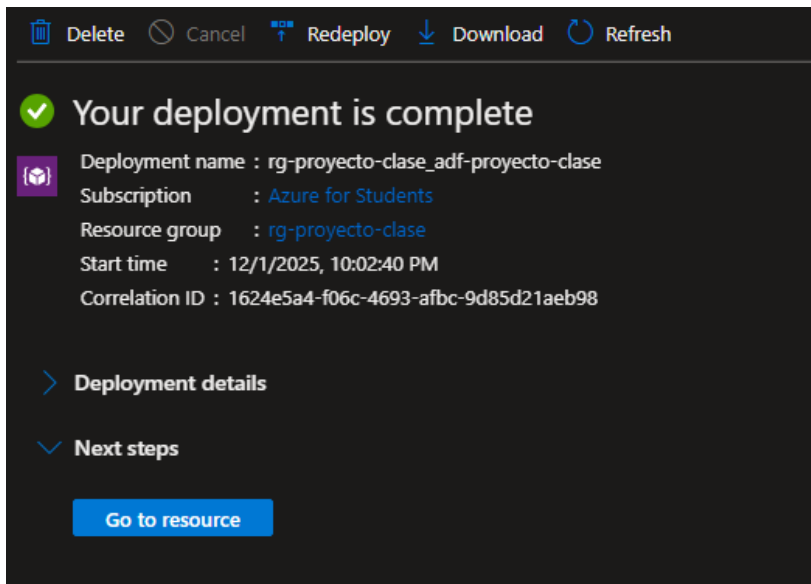
1. En el portal de Azure, buscar "Azure Databricks".
2. Pulsar en "+ Crear".



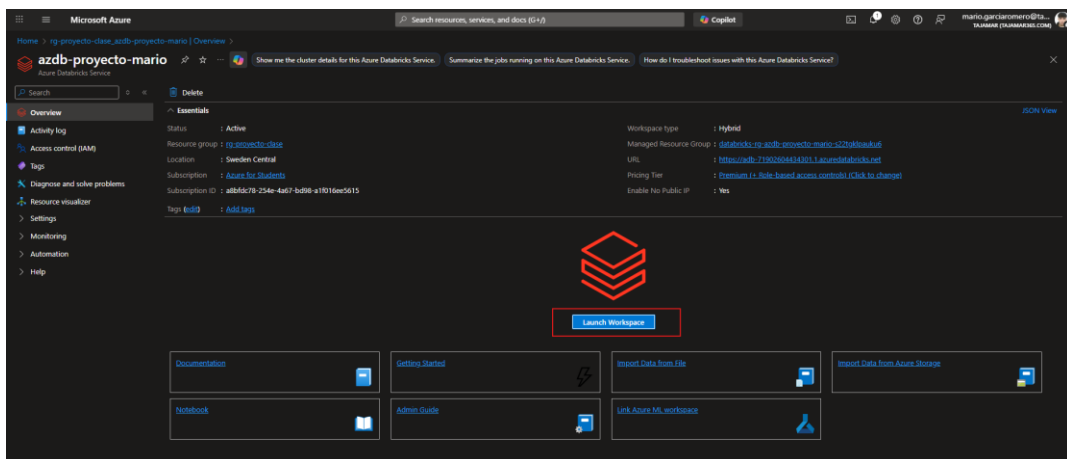
3. Rellenar:

- Suscripción y grupo de recursos (rg-proyecto-clase).
- Nombre del workspace: por ejemplo, adb-proyecto-clase.
- Ubicación: igual que el resto de los recursos o cerca, en este caso no deja crear en Spain central, por lo que elegimos sweden central.
- Workspace type: Hybrid (serverless da error)

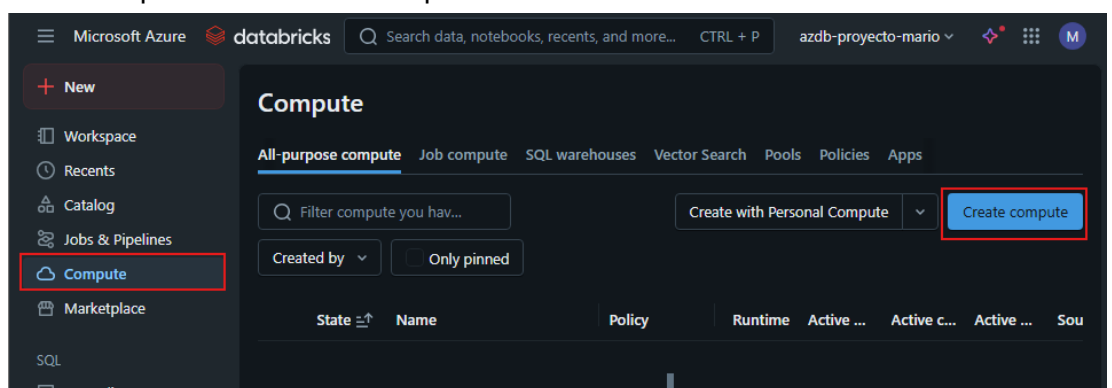
4. Hacer clic en “Revisar y crear” y luego “Crear”.



5. Cuando el workspace esté listo, entrar al recurso y pulsar “Launch Workspace” para abrir la interfaz de Databricks.



6. En Databricks:
 - a. Ir a “Compute” → “+ New Compute”.



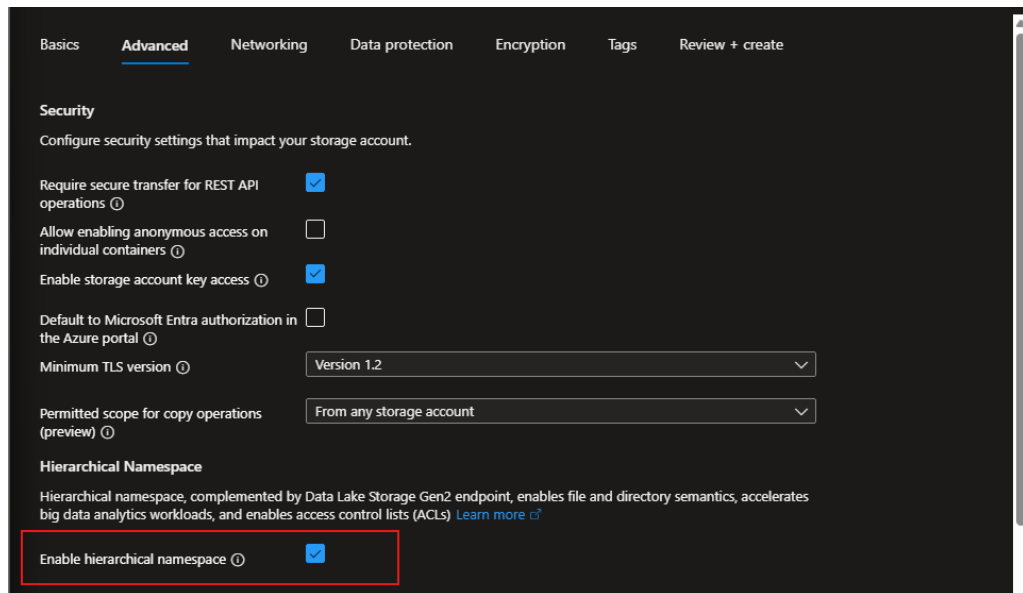
- b. Crear un clúster de tipo “All-purpose” (para desarrollo), con un tamaño pequeño para pruebas, node type Standard_D4s_v3 por ejemplo
 - c. Asignar un nombre, por ejemplo, cluster-proyecto.

d. Iniciar el clúster.

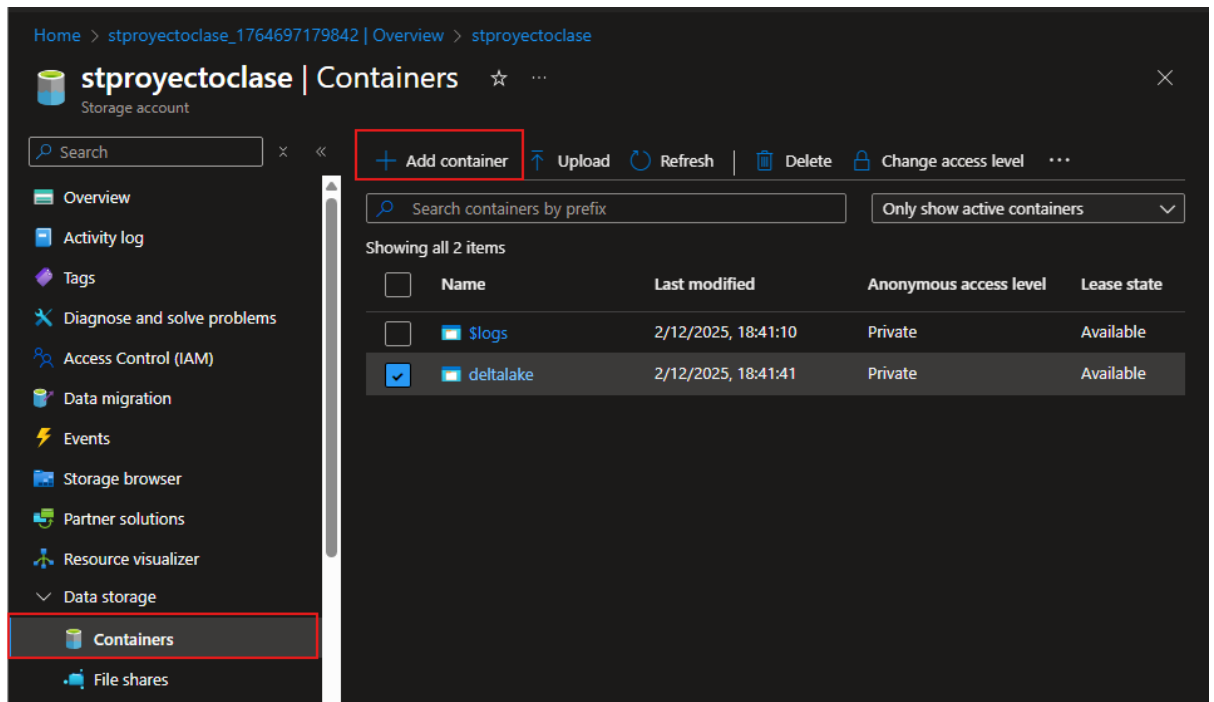
2.4. Configurar almacenamiento para Delta Lake

1. En Azure Portal, crear una cuenta de almacenamiento:
 - a. Buscar “Storage account” → “+ Crear”.
 - b. Nombre: stproyectoclase.
 - c. Región: la misma que los demás recursos.

d. Habilitar “Hierarchical namespace” (En advanced).



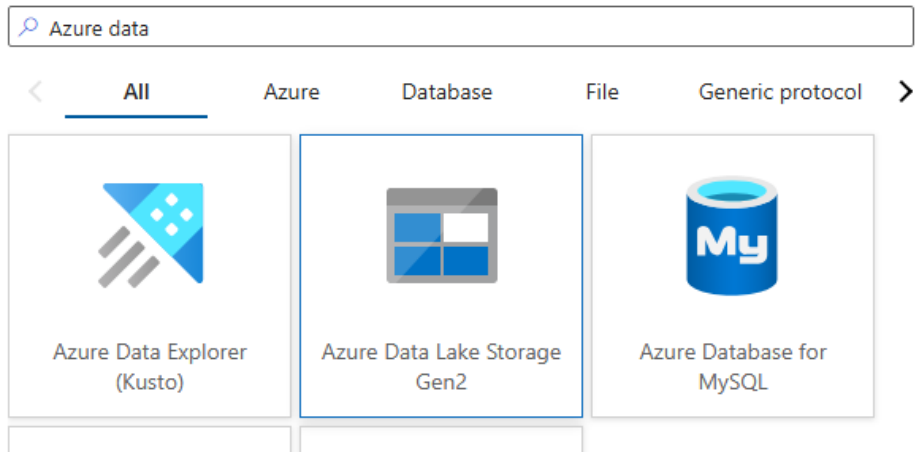
2. Una vez creada la cuenta de almacenamiento, ir a “Containers” y crear un contenedor, por ejemplo, datalake.



3. Tomar nota del nombre de la cuenta y del contenedor (se usarán en Databricks y ADF), nombre: stprojectoclase, contenedor: datalake.

2.5.2. Linked Service a Azure Data Lake Storage Gen2 (destino / Delta Lake)

1. En “Manage” → “Linked services” → “+ New”.
2. Seleccionar el conector “Azure Data Lake Storage Gen2”.



3. Configurar:

- Nombre: LS_ADLS_StProyectoClase.
- Storage account: seleccionar la cuenta stproyectoclase.
- Método de autenticación: “Account key” (clave de la cuenta de almacenamiento).

4. Pulsar “Test connection”.

New linked service
 Azure Data Lake Storage Gen2 [Learn more](#)

Name *

Description

Connect via integration runtime *
☒ AutoResolveIntegrationRuntime

Authentication type

Account selection method
☒ From Azure subscription ☐ Enter manually

Azure subscription

Storage account name *

Test connection
☒ To linked service ☐ To file path

Annotations
[+ New](#)

[Parameters](#)

[Advanced](#)

[Create](#) [Back](#)

✓ Connection successful
[Test connection](#) [Cancel](#)

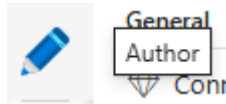
5. Pulsar “Create”.

2.6 Crear datasets de origen y destino

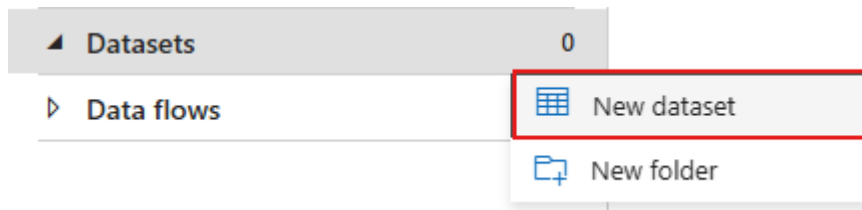
2.6.1. Datasets de origen (Azure SQL)

1. Dataset de origen (Azure SQL):

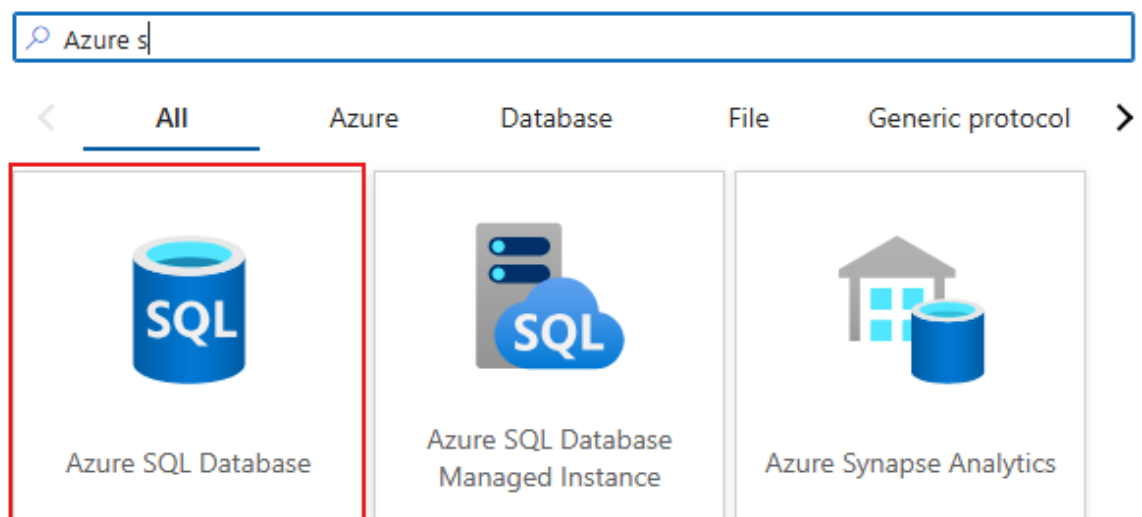
a. En ADF, ir a “Author” (icono del lápiz).



b. En la sección “Datasets”, pulsar en “+ New dataset”.



c. Elegir “Azure SQL Database”.



d. Darle un nombre, por ejemplo, DS_SQL_Ventas.

e. Seleccionar el Linked Service LS_AzureSQL_Origen.

f. Elegir la tabla, por ejemplo, dbo.Ventas.

En el caso de que al intentar seleccionar la tabla de error, deberemos ir a el linked service de azure sql y darle a probar la conexión, si sale la conexión correcta, volvemos a realizar este paso y deberán salir

Set properties

Name
DS_SQL_Ventas

Linked service *
LS_AzureSQL_Origen

Table name
dbo.Ventas
☐ Enter manually

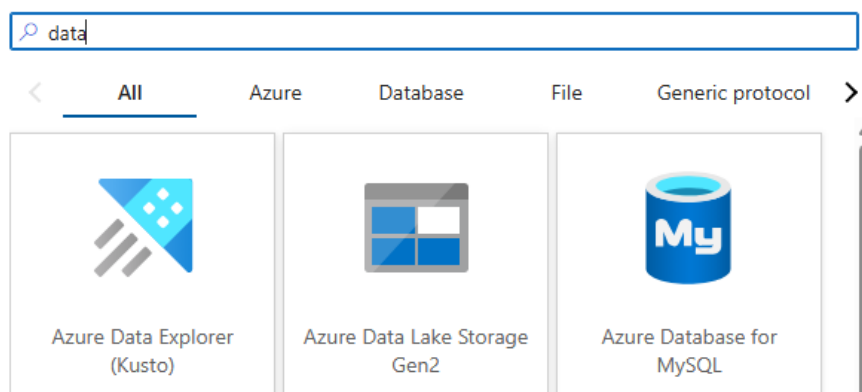
Import schema
☒ From connection/store ☐ None

Repetimos el proceso creando un dataset por cada tabla de la base de datos

2.6.2. Datasets de destino (Delta Lake en ADLS Gen2)

Para cada tabla se crea un dataset que apunta al contenedor y carpeta donde se almacenarán los datos en el Data Lake. ADF escribirá los datos en formato de fichero (Parquet/Delta) y esas rutas serán posteriormente leídas desde Databricks.

1. En “Datasets”, pulsar “+ New dataset”.
2. Seleccionar el conector “Azure Data Lake Storage Gen2” y, como formato de archivo, “Parquet”.



Select format

Choose the format type of your data

 Avro	 Binary	 DelimitedText
 Excel	 Iceberg	 JSON
 ORC	 Parquet	 XML

3. En Set properties:

- Nombre:
 - DS_Delta_Clientes, DS_Delta_Productos o DS_Delta_Ventas, según la tabla.
- Linked service: LS_ADLS_StProyectoClase.
- File Path: deltalake / bronze/clientes(o la tabla que estemos haciendo) / clientes.parquet(o el nombre que se quiera)
- Import schema: None.
- Pulsar OK.

Set properties

Name

Linked service *

File path

/

/



Import schema

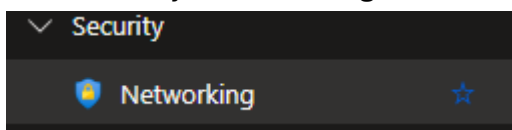
☐ From connection/store ☐ From sample file ☒ None

Si las carpetas bronze/clientes, bronze/productos o bronze/ventas no existen todavía, ADF las creará automáticamente al ejecutar por primera vez la actividad de copia hacia esas rutas.

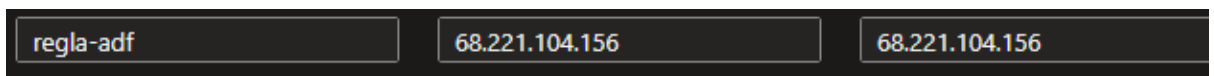
2.7. Crear pipeline de copia de Azure SQL a Delta Lake

2.7.1 Añadir regla de firewall

1. En Azure Portal, abre el servidor lógico srv-proyecto-clase.
2. Ve a Security → Networking.



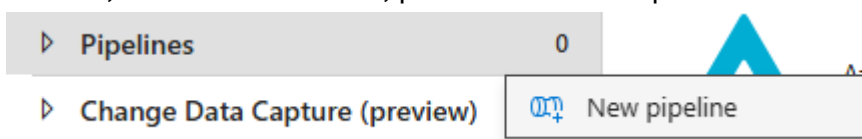
3. En la pestaña Public access, asegúrate de que Public network access = Selected networks.
4. En Firewall rules, crea una nueva regla:
 - a. Name: regla-adf-2.
 - b. Start IP y End IP: 68.221.104.183 (la IP que sale en el error).



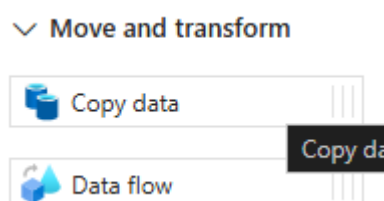
5. Guarda y espera 2–3 minutos.
6. Vuelve a ADF

2.7.2 Crear pipeline

1. En ADF, dentro de “Author”, pulsar en “+” → “Pipeline”.



2. Asignar un nombre al pipeline, por ejemplo, SQL_Delta.
3. En el panel de actividades, arrastrar una actividad “Copy data” al lienzo.



4. Configurar la actividad:
 - a. Pestaña “Source”:

i. Dataset: DS_SQL_Ventas.

General **Source** Sink¹ Mapping Settings User properties

Source dataset * DS_SQL_Ventas

[Open](#) [+ New](#) [Preview data](#) [Learn more](#)

Use query ☒ Table ☐ Query ☐ Stored procedure

Query timeout (minutes) ⓘ

Isolation level ⓘ Select...

Partition option ⓘ ☒ None ☐ Physical partitions of table ⓘ ☐ Dynamic range ⓘ

i Please preview data to validate the partition settings.

Additional columns ⓘ [+ New](#)

b. Pestaña “Sink”:

i. Dataset: DS_Delta_Ventas.

General Source **Sink** Mapping Settings User properties

Sink dataset * DS_Delta_Ventas [Open](#) [+ New](#) [Learn more](#)

Copy behavior ⓘ Select...

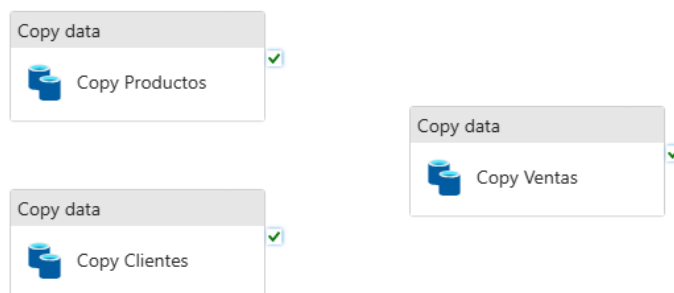
Max concurrent connections ⓘ

Block size (MB) ⓘ

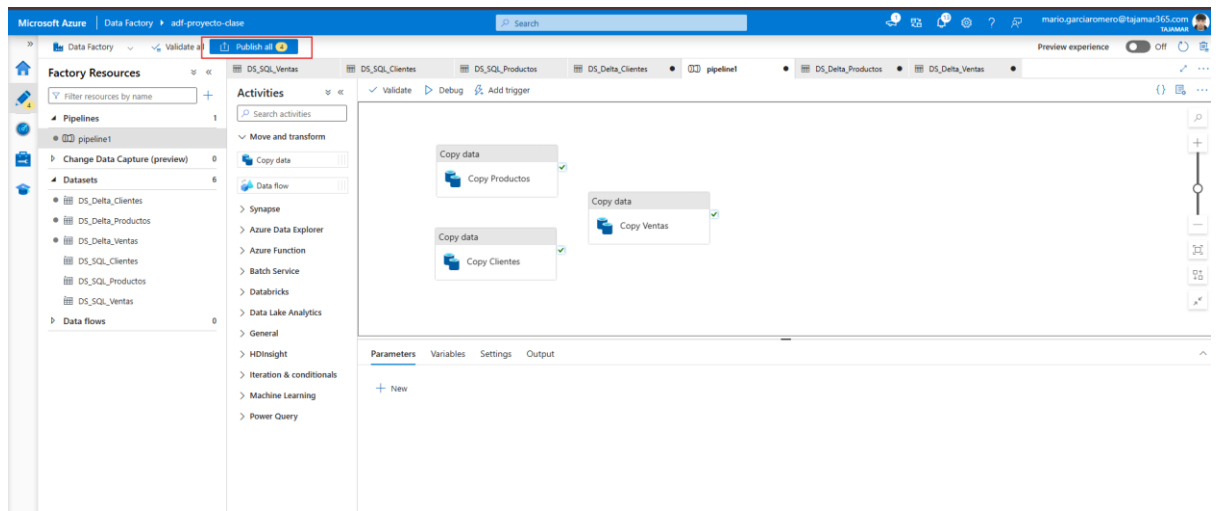
Metadata ⓘ [+ New](#)

Max rows per file ⓘ

5. Realizar el proceso en el mismo pipeline, un copy data por cada tabla



6. Guardar y pulsar “Publish all” para publicar el pipeline.



Publish all

You are about to publish all pending changes to the live environment. [Learn more](#)

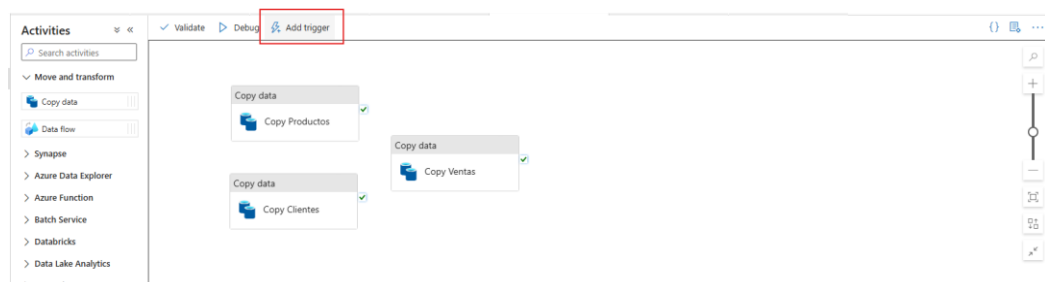
Pending changes (4)

NAME	CHANGE	EXISTING
Pipelines		
pipeline1	(New)	-
Datasets		
DS_Delta_Clientes	(Edited)	DS_Delta_Clientes
DS_Delta_Productos	(New)	-
DS_Delta_Ventas	(New)	-

[Publish](#) [Cancel](#)

7. Ejecutar el pipeline:

- Hacer clic en “Add trigger” → “Trigger now”.



- Confirmar y ejecutar.

8. Ir a la pestaña “Monitor” en ADF y comprobar que la ejecución del pipeline termina correctamente (estado “Succeeded”).

☐	pipeline1		12/2/2025, 8:45:28 PM	12/2/2025, 8:45:58 PM	31s	Manual trigger		Succeeded	Original	039ed566-cafe-4563-9b63-a4d9e3e56f
--------------------------	-----------	--	-----------------------	-----------------------	-----	----------------	--	-----------	----------	------------------------------------

Desde donde estás ahora, documenta Databricks como “Lakehouse” registrando tablas sobre el Delta/Parquet del Storage, y luego las conexiones de consumo.

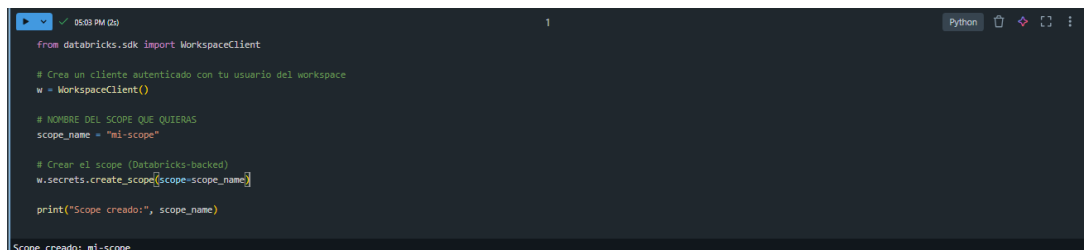
4. Lakehouse en Databricks a partir del Data Lake

4.1. Configurar el scope en databricks

¿Qué es un scope en Databricks?

Un secret scope es como una carpeta con nombre donde guardas secretos (contraseñas, claves, tokens) de forma segura dentro de Databricks.

- El *scope* tiene un nombre (por ejemplo, mi-scope).
 - Dentro van secretos en formato clave-valor:
 - adls-key → la clave del storage
 - db-user → usuario de BD
 - db-pass → contraseña
- 1) Sube a databricks el notebook ConfiguracionScopes y vas ejecutando las celdas, a continuación, se explica el notebook
- a) Creamos el scope, aqui puedes cambiar el nombre al que quieras



```
from databricks.sdk import WorkspaceClient

# Crea un cliente autenticado con tu usuario del workspace
w = WorkspaceClient()

# NOMBRE DEL SCOPE QUE QUIERAS
scope_name = "mi-scope"

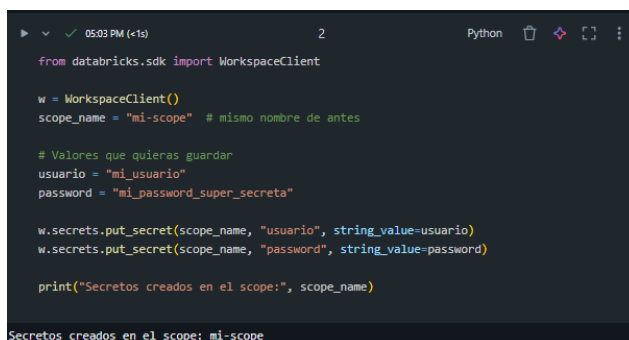
# Crear el scope (Databricks-backed)
w.secrets.create_scope(scope_name)

print("Scope creado:", scope_name)
```

Scope creado: mi-scope

- Esta celda inicializa el cliente de Databricks (WorkspaceClient) y crea un nuevo secret scope llamado mi-scope dentro del workspace.
- Es el paso previo para poder almacenar secretos; solo se ejecuta una vez por scope.

- b) Configuramos el scope, obviamente puedes cambiar tanto la contraseña como el nombre de usuario



```
from databricks.sdk import WorkspaceClient

w = WorkspaceClient()
scope_name = "mi-scope" # mismo nombre de antes

# Valores que quieras guardar
usuario = "mi_usuario"
password = "mi_password_super_secreta"

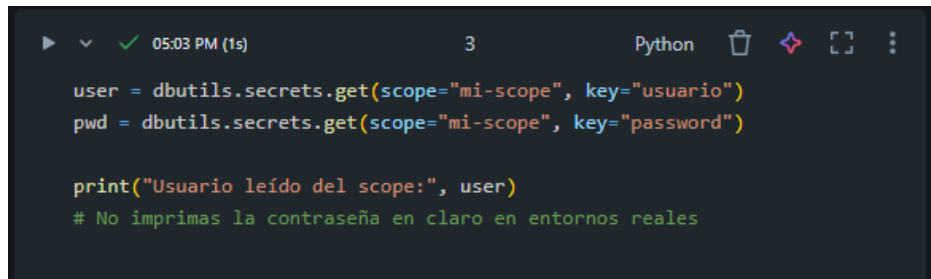
w.secrets.put_secret(scope_name, "usuario", string_value=usuario)
w.secrets.put_secret(scope_name, "password", string_value=password)

print("Secretos creados en el scope:", scope_name)
```

Secretos creados en el scope: mi-scope

- Esta celda usa el scope creado antes (mi-scope) para guardar dos secretos: uno con la clave usuario y otro con la clave password.
- Los valores se almacenan cifrados en Databricks y luego se podrán recuperar desde notebooks sin exponerlos en claro en el código.

c) Leemos y comprobamos los datos del scope



```

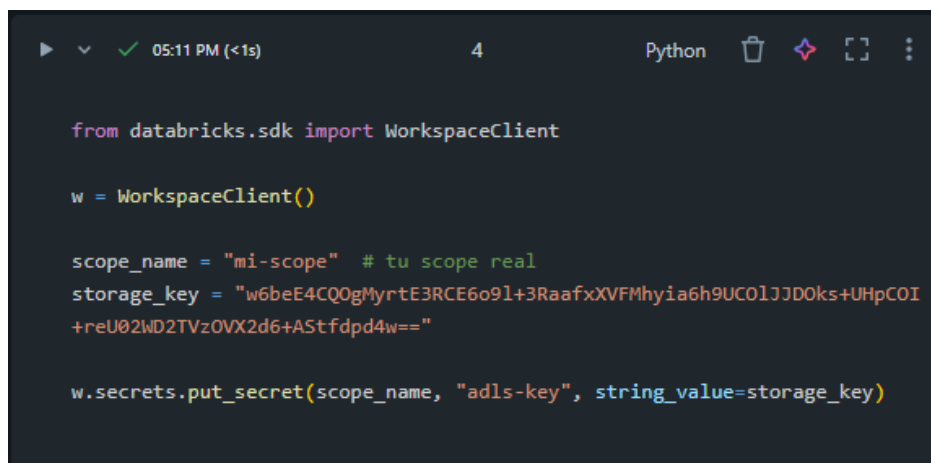
05:03 PM (1s) 3 Python
user = dbutils.secrets.get(scope="mi-scope", key="usuario")
pwd = dbutils.secrets.get(scope="mi-scope", key="password")

print("Usuario leído del scope:", user)
# No imprimas la contraseña en claro en entornos reales

```

- Esta celda recupera los valores almacenados en el scope mi-scope usando dbutils.secrets.get.
- Sirve para comprobar que los secretos se han guardado correctamente y para usarlos después en conexiones (por ejemplo, cadenas JDBC, credenciales, etc.).

d) Guardar la clave del storage (adls-key)



```

05:11 PM (<1s) 4 Python
from databricks.sdk import WorkspaceClient

w = WorkspaceClient()

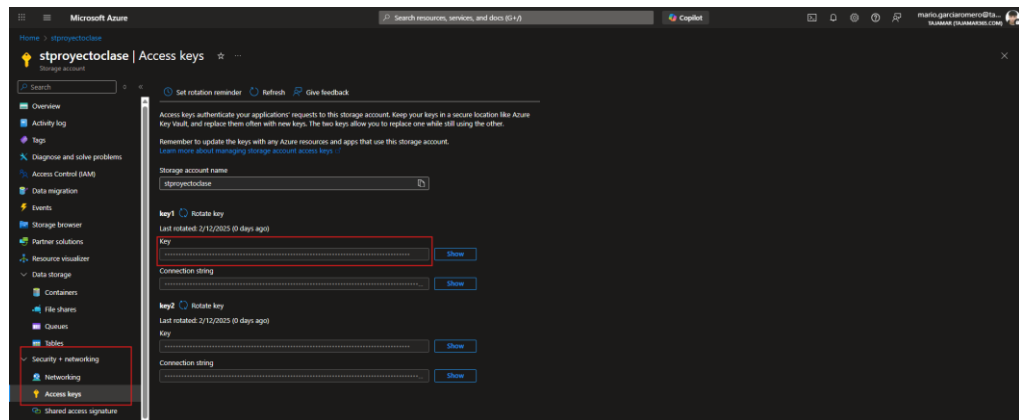
scope_name = "mi-scope" # tu scope real
storage_key = "w6beE4CQ0gMyrtE3RCE6o9l+3RaafxXVFMhya6h9UC0lJJJD0ks+UHpCOI
+reU02WD2TVzOVX2d6+AStfdpd4w=="

w.secrets.put_secret(scope_name, "adls-key", string_value=storage_key)

```

- Esta celda guarda en el mismo scope mi-scope un tercer secreto llamado adls-key que contiene la access key de la cuenta de almacenamiento de Azure (ADLS).
- Ese secreto se usa después en las lecturas/escrituras a ADLS, por ejemplo con dbutils.secrets.get(scope="mi-scope", key="adls-key") en las opciones de conexión, evitando poner la clave del storage en texto plano.

Para ver esa storage_key, debemos irnos al storage account, Security + networking → Access keys, le das a show en alguna de las dos keys y copias y pegas en la celda donde pone storage_key, poniendo la tuya



4.1. Leer datos del Data Lake en Databricks

Una vez tenemos el scope configurado, ya podemos leer las tablas dentro de nuestro container

- 1) para ello deberemos crear otro notebook y para acceder a las tablas poner esto:

```
df_clientes = spark.read.format("parquet").option(
    "fs.azure.account.key.stproyectoclase.dfs.core.windows.net",
    dbutils.secrets.get(scope="mi-scope", key="adls-key")
).load(
    "abfss://deltalake@stproyectoclase.dfs.core.windows.net/bronze/clientes/"
)
```

Aquí obviamente sustituyendo la ruta por la ruta que tengas tu y el nombre del scope, deberemos de hacer esa lectura por cada una de las tablas que tengamos (archivos .parquet)

```
DS:15 PM (1s) 2 Python
#Tabla clientes
df_clientes = spark.read.format("parquet").option(
    "fs.azure.account.key.stprojectoclase.dfs.core.windows.net",
    dbutils.secrets.get(scope="mi-scope", key="adls-key")
).load(
    "abfss://deltalake@stprojectoclase.dfs.core.windows.net/bronze/clientes/"
)

#Tabla productos
df_productos = spark.read.format("parquet").option(
    "fs.azure.account.key.stprojectoclase.dfs.core.windows.net",
    dbutils.secrets.get(scope="mi-scope", key="adls-key")
).load(
    "abfss://deltalake@stprojectoclase.dfs.core.windows.net/bronze/productos/"
)

#Tabla ventas
df_ventas = spark.read.format("parquet").option(
    "fs.azure.account.key.stprojectoclase.dfs.core.windows.net",
    dbutils.secrets.get(scope="mi-scope", key="adls-key")
).load(
    "abfss://deltalake@stprojectoclase.dfs.core.windows.net/bronze/ventas/"
)

df_clientes: pyspark.sql.connect.dataframe.DataFrame = [IdCliente: integer, Nombre: string ... 4 more fields]
```

2) Comprobamos que se han cargado datos:

df_clientes.show(5)

df_productos.show(5)

df_ventas.show(5)

```
Just now (1s) 3
df_clientes.show(5)
df_ventas.show(5)
df_clientes.show(5)
> See performance \(3\)
```

IdCliente	Nombre	Segmento	Ciudad	Pais	FechaAlta
1	Cliente A	Retail	Madrid	España	2024-01-10
2	Cliente B	Empresa	Barcelona	España	2024-02-15
3	Cliente C	Online	Valencia	España	2024-03-20
4	Cliente D	Retail	Sevilla	España	2024-04-05
5	Cliente E	Empresa	Bilbao	España	2024-05-12

IdVenta	Fecha	IdCliente	IdProducto	Cantidad	PrecioUnitario	ImporteTotal
1	2025-01-05	1	1	1	800.00	800.00
2	2025-01-10	2	3	1	150.00	150.00
3	2025-01-12	3	2	2	25.00	50.00

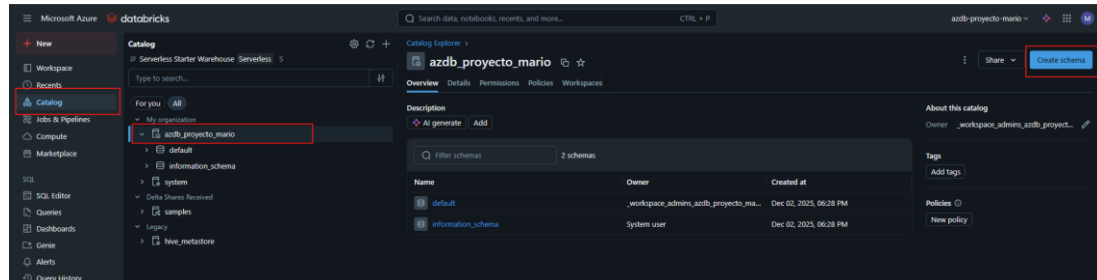
IdCliente	Nombre	Segmento	Ciudad	Pais	FechaAlta
1	Cliente A	Retail	Madrid	España	2024-01-10
2	Cliente B	Empresa	Barcelona	España	2024-02-15
3	Cliente C	Online	Valencia	España	2024-03-20
4	Cliente D	Retail	Sevilla	España	2024-04-05
5	Cliente E	Empresa	Bilbao	España	2024-05-12

Con esto, Databricks consume el Delta Lake físico almacenado en ADLS Gen2.

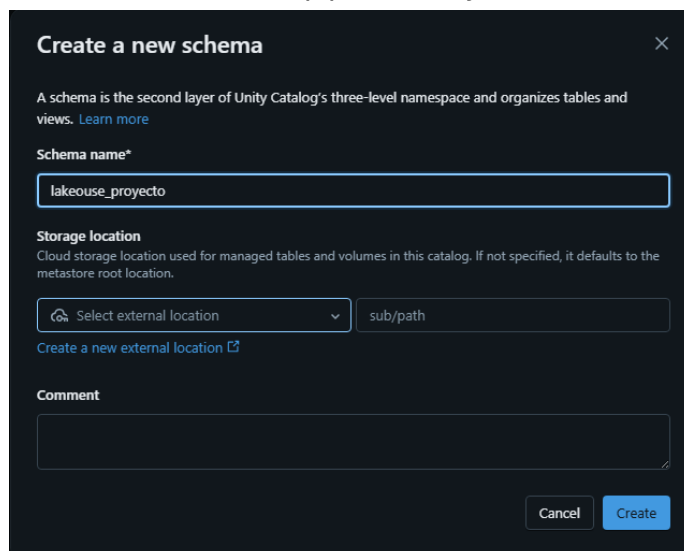
4.2. Crear tablas gestionadas en Databricks (Lakehouse lógico)

1. Creamos un catalog:

- Vamos a catalog en el menú de la izquierda
- Seleccionamos el catalog nuestro y create schema



iii. Ponemos el nombre q queramos y le damos a crear



Create a new schema

A schema is the second layer of Unity Catalog's three-level namespace and organizes tables and views. [Learn more](#)

Schema name*

lakeouse_proyecto

Storage location

Cloud storage location used for managed tables and volumes in this catalog. If not specified, it defaults to the metastore root location.

Select external location sub/path

[Create a new external location](#)

Comment

Cancel Create

2. Guardar los DataFrames como **tablas gestionadas** (managed tables) en el schema que acabamos de crear.

En el mismo notebook de antes, ejecutamos esta celda (cambiando el nombre del catálogo por el que tengas tu y lo mismo con el schema):

```
# Guardar datos como tablas gestionadas (Lakehouse lógico)
```

```
#Clientes
```

```
df_clientes.write.format("delta") \
```

```
.mode("overwrite") \
```

```
.saveAsTable("azdb_proyecto_mario.lakeouse_proyecto.delta_clientes")
```


#Productos

```
df_productos.write.format("delta") \

.mode("overwrite") \

.saveAsTable("azdb_proyecto_mario.lakeouse_proyecto.delta_productos")
```

#Ventas

```
df_ventas.write.format("delta") \

.mode("overwrite") \

.saveAsTable("azdb_proyecto_mario.lakeouse_proyecto.delta_ventas")
```

A screenshot of a Databricks notebook interface. The top bar shows a play button, a checkmark, and the text 'Just now (5s)'. The notebook content is a Python script with three sections: '#Clientes', '#Productos', and '#Ventas'. Each section contains code to write data to a Delta table in a specific schema. The code for '#Productos' is highlighted. The bottom of the notebook shows a link to 'See performance (3)'.

```
# Guardar datos como tablas gestionadas (Lakehouse lógico)
#Clientes
df_clientes.write.format("delta") \
    .mode("overwrite") \
    .saveAsTable("azdb_proyecto_mario.lakeouse_proyecto.delta_clientes")

#Productos
df_productos.write.format("delta") \
    .mode("overwrite") \
    .saveAsTable("azdb_proyecto_mario.lakeouse_proyecto.delta_productos")

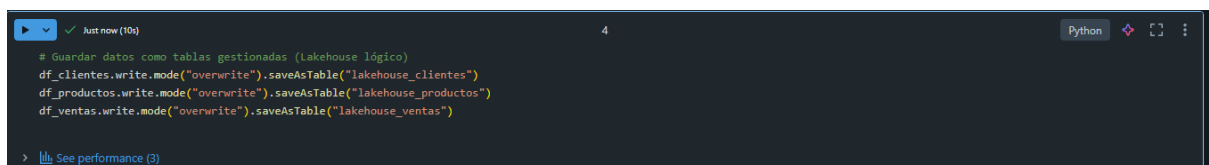
#Ventas
df_ventas.write.format("delta") \
    .mode("overwrite") \
    .saveAsTable("azdb_proyecto_mario.lakeouse_proyecto.delta_ventas")

> See performance \(3\)
```

3. En el notebook

Guardar datos como tablas gestionadas (Lakehouse lógico)

```
df_clientes.write.mode("overwrite").saveAsTable("lakehouse_clientes")
df_productos.write.mode("overwrite").saveAsTable("lakehouse_productos")
df_ventas.write.mode("overwrite").saveAsTable("lakehouse_ventas")
```

A screenshot of a Databricks notebook interface. The top bar shows a play button, a checkmark, and the text 'Just now (10s)'. The notebook content is a Python script with a single section: '# Guardar datos como tablas gestionadas (Lakehouse lógico)'. It contains code to write data to three Lakehouse tables. The code is highlighted. The bottom of the notebook shows a link to 'See performance (3)'.

```
# Guardar datos como tablas gestionadas (Lakehouse lógico)
df_clientes.write.mode("overwrite").saveAsTable("lakehouse_clientes")
df_productos.write.mode("overwrite").saveAsTable("lakehouse_productos")
df_ventas.write.mode("overwrite").saveAsTable("lakehouse_ventas")

> See performance \(3\)
```

2. Comprobar que existen las tablas en el Lakehouse de Databricks:

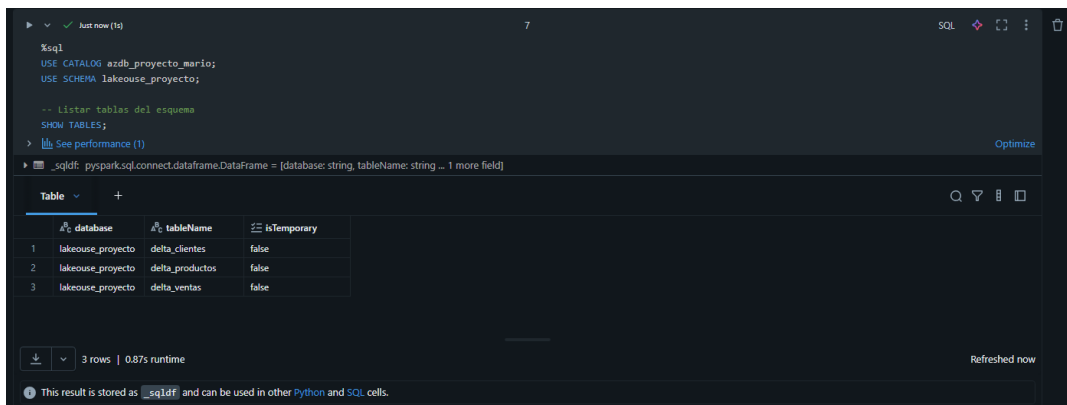
%sql

USE CATALOG azdb_proyecto_mario; --(cambia nombre)

USE SCHEMA lakehouse_proyecto; --(cambia nombre)

-- Listar tablas del esquema

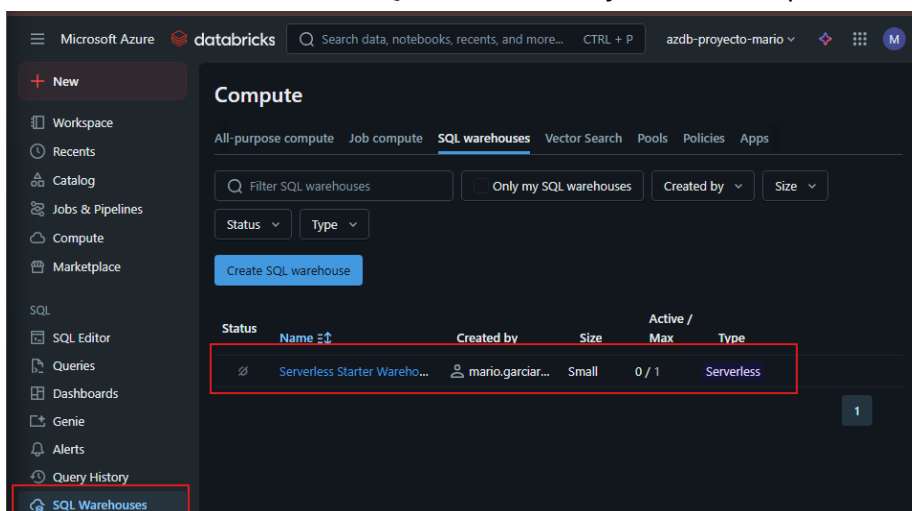
SHOW TABLES;



Estas tablas representan la **capa Lakehouse** gestionada por Databricks, construida a partir del Data Lake en ADLS y se almacenan en el catalog en el schema creado.

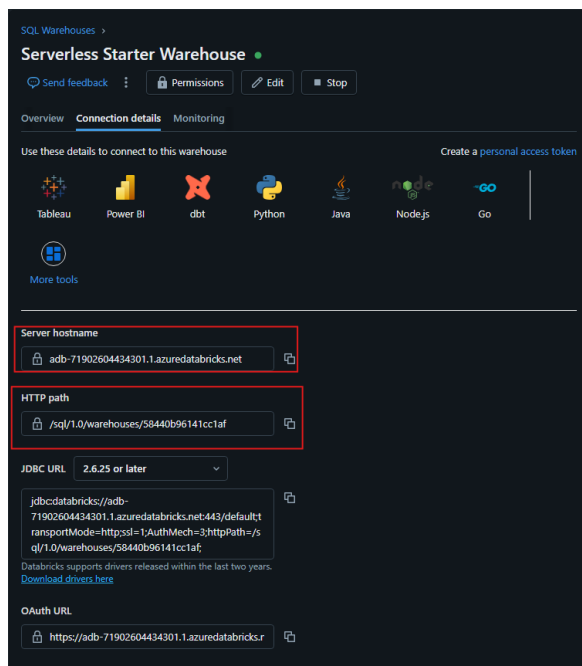
5. Conexión de Power BI al Lakehouse (Databricks)

1. Power BI → Databricks (tus tablas UC)
2. En Azure Databricks, ve a SQL Warehouses y usamos el que se nos crea solo.



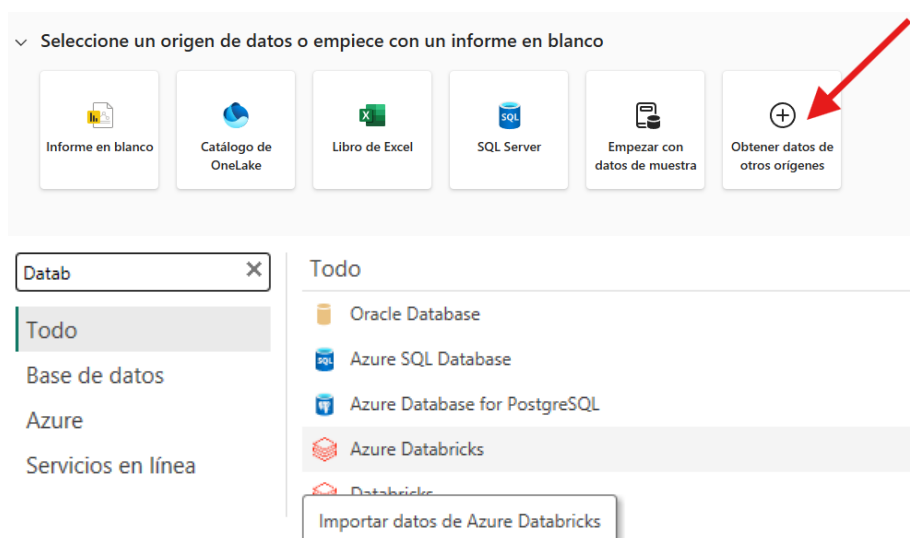
Ve a Connection details y anota:

- a. Server hostname
- b. HTTP path



3. En Power BI Desktop:

- a. Obtener datos → Azure → Azure Databricks.



- b. Pon el *server hostname* y el *HTTP path* y le damos a Aceptar

Azure Databricks

Nombre de host del servidor ⓘ

Ruta de acceso HTTP ⓘ

⚡ Opciones avanzadas (opcional)

Catálogo predeterminado (opcional) ⓘ

Base de datos (opcional) ⓘ

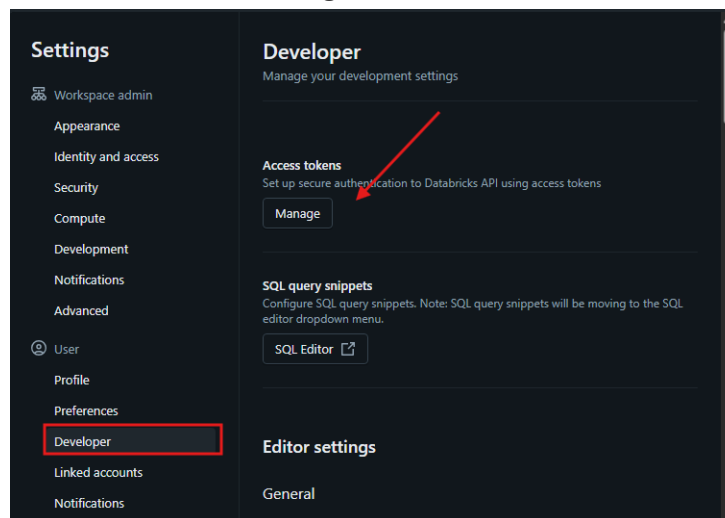
Query tags (opcional) ⓘ

Detección automática de proxy (opcional) ⓘ

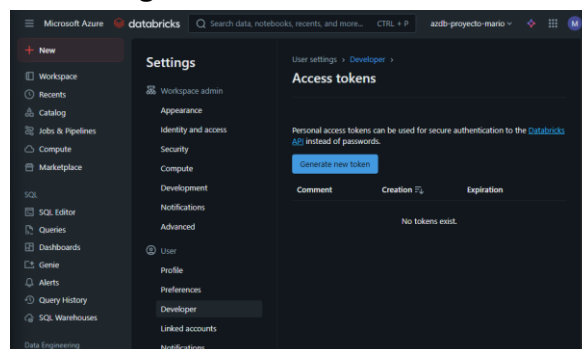
Implementación (opcional) ⓘ

Consulta nativa (Requiere: Catálogo predeterminado... (opcional) ⓘ)

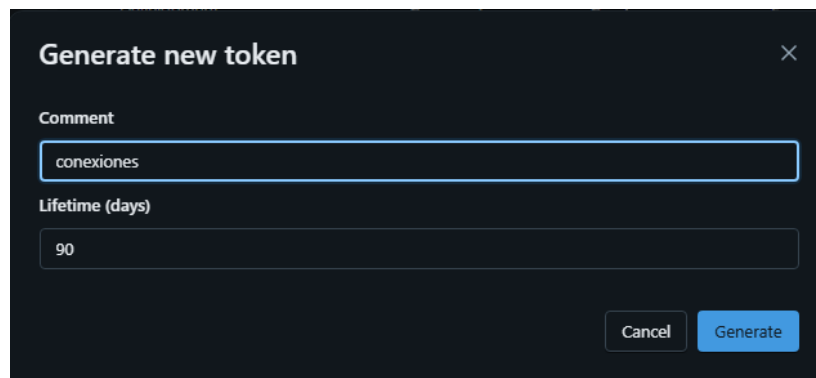
- c. Autenticación: Necesitaremos crear un token en databricks, para ello nos vamos a nuestro perfil (arriba a la derecha) → Settings → Developer → En acces tokens manage



Dentro → generate new token



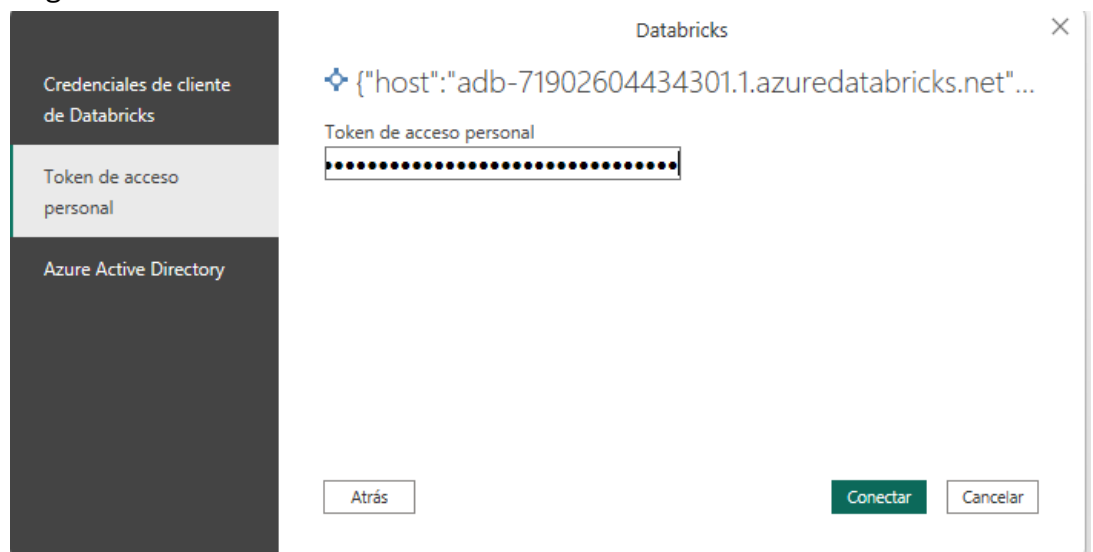
y le ponemos el uso y a generate



A dark-themed dialog box titled "Generate new token" with a close button (X) in the top right corner. It contains two input fields: "Comment" with the text "conexiones" and "Lifetime (days)" with the text "90". At the bottom right are two buttons: "Cancel" and "Generate".

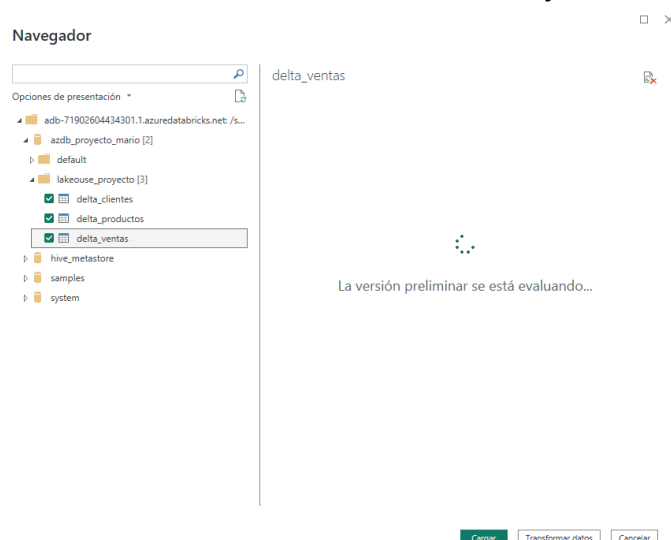
COPIAD EL TOKEN Y GUARDARLO PORQUE LUEGO PASA LO Q PASA

Pegamos el token en Power BI:



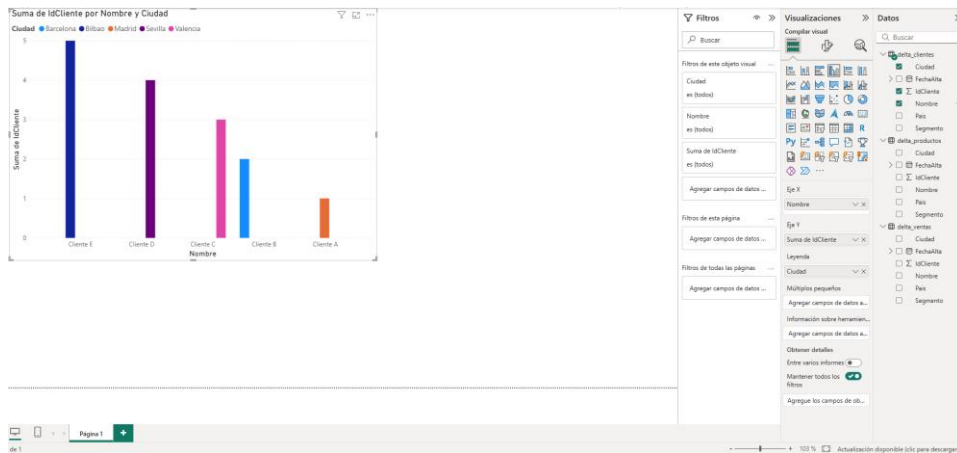
A window titled "Databricks" with a close button (X) in the top right corner. On the left is a sidebar with three sections: "Credenciales de cliente de Databricks", "Token de acceso personal" (highlighted), and "Azure Active Directory". The main area shows a Databricks logo, a host URL in a code block: `{"host": "adb-71902604434301.1.azuredatabricks.net"...`, and a "Token de acceso personal" field filled with dots. At the bottom are three buttons: "Atrás", "Conectar", and "Cancelar".

4. En el navegador de tablas de Power BI verás tu catálogo/esquema, por ejemplo: azdb_proyecto_mario → lakeouse_proyecto → delta_clientes, delta_productos, delta_ventas. Seleccionamos las tablas y le damos a cargar



A screenshot of the Power BI interface. On the left is the "Navegador" (Navigator) pane showing a tree structure of data sources. The tree includes "adb-71902604434301.1.azuredatabricks.net / s...", "azdb_proyecto_mario [2]", "default", "lakeouse_proyecto [3]", "delta_clientes", "delta_productos", and "delta_ventas" (which is selected and highlighted). Below the tree are "hive_metastore", "samples", and "system". On the right, the main area shows the selected table "delta_ventas" with a loading spinner and the text "La versión preliminar se está evaluando...". At the bottom are three buttons: "Cargar", "Transformar datos", and "Cancelar".

Y ya podríamos acceder a las tablas para poder hacer reportes



Tu flujo de Databricks ML se puede documentar así, paso a paso y ligado a tus tablas delta_clientes, delta_productos y delta_ventas del lakehouse.

6. Uso del lakehouse como origen de datos en Databricks ML

En un notebook específico de Machine Learning se utilizan directamente las tablas Delta creadas en el catálogo azdb_proyecto_mario, esquema lakeouse_proyecto. Estas tablas (delta_clientes, delta_productos, delta_ventas) forman el lakehouse del proyecto y se consumen con spark.table, sin necesidad de copias adicionales de datos.

Código de lectura:

```
python
spark.sql("USE CATALOG azdb_proyecto_mario")
spark.sql("USE SCHEMA lakeouse_proyecto")

df_clientes = spark.table("azdb_proyecto_mario.lakeouse_proyecto.delta_clientes")
df_productos =
spark.table("azdb_proyecto_mario.lakeouse_proyecto.delta_productos")
df_ventas = spark.table("azdb_proyecto_mario.lakeouse_proyecto.delta_ventas")

df_ventas.show(5)
```

```
▶ Just now (2s) 1 Python
spark.sql("USE CATALOG azdb_proyecto_mario")
spark.sql("USE SCHEMA lakeouse_proyecto")

df_clientes = spark.table("azdb_proyecto_mario.lakeouse_proyecto.
delta_clientes")
df_productos = spark.table("azdb_proyecto_mario.lakeouse_proyecto.
delta_productos")
df_ventas = spark.table("azdb_proyecto_mario.lakeouse_proyecto.
delta_ventas")

df_ventas.show(5)

> See performance (3)
df_clientes: pyspark.sql.connect.dataframe.DataFrame = [IdCliente: integer, Nombre: string
... 4 more fields]
df_productos: pyspark.sql.connect.dataframe.DataFrame = [IdCliente: integer, Nombre:
string ... 4 more fields]
df_ventas: pyspark.sql.connect.dataframe.DataFrame = [IdCliente: integer, Nombre: string ...
4 more fields]

+-----+-----+-----+-----+-----+-----+
|IdCliente| Nombre|Segmento| Ciudad| Pais| FechaAlta|
+-----+-----+-----+-----+-----+
| 1|Cliente A| Retail| Madrid| España| 2024-01-10|
| 2|Cliente B| Empresa| Barcelona| España| 2024-02-15|
| 3|Cliente C| Online| Valencia| España| 2024-03-20|
| 4|Cliente D| Retail| Sevilla| España| 2024-04-05|
| 5|Cliente E| Empresa| Bilbao| España| 2024-05-12|
+-----+-----+-----+-----+-----+

```

6.1. Preparación sencilla de datos (ventas por cliente)

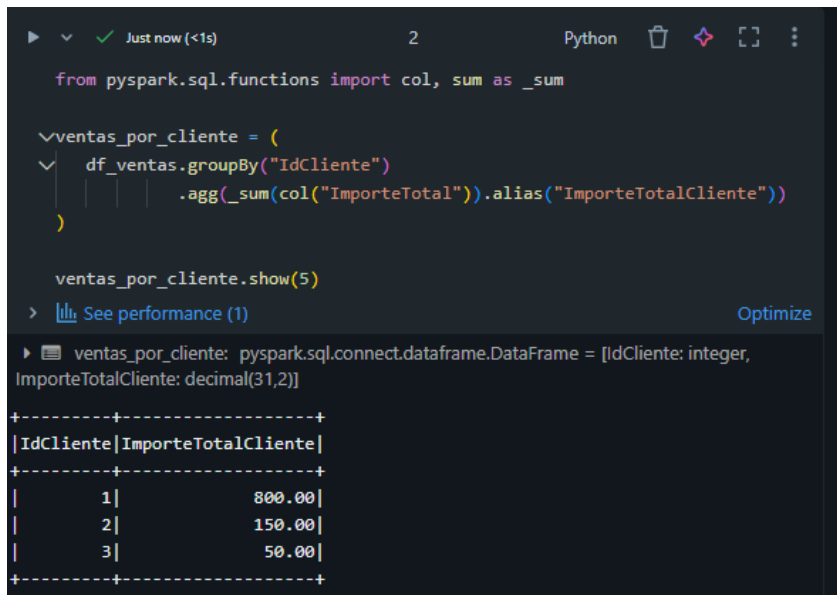
A partir de la tabla de ventas (delta_ventas) se construye un conjunto de datos muy simple para Machine Learning: el importe total acumulado por cliente. De este modo se genera una fila por cliente con su gasto total, que servirá como base para un modelo predictivo de ejemplo.

Código de agregación:

```
from pyspark.sql.functions import col, sum as _sum
```

```
ventas_por_cliente = (
    df_ventas.groupBy("IdCliente")
        .agg(_sum(col("ImporteTotal")).alias("ImporteTotalCliente"))
)
```

```
ventas_por_cliente.show(5)
```



The screenshot shows a Databricks notebook interface. At the top, it says 'Just now (<1s)' and '2' lines of code. The code is in Python and uses PySpark SQL functions. It defines a variable 'ventas_por_cliente' as a DataFrame created by grouping 'df_ventas' by 'IdCliente' and aggregating the sum of 'ImporteTotal' as 'ImporteTotalCliente'. Then, it calls 'ventas_por_cliente.show(5)'. Below the code, there is a link 'See performance (1)' and an 'Optimize' button. The output of the show command is displayed as a table with two columns: 'IdCliente' and 'ImporteTotalCliente'. The table shows three rows of data.

```
from pyspark.sql.functions import col, sum as _sum

ventas_por_cliente = (
    df_ventas.groupBy("IdCliente")
    .agg(_sum(col("ImporteTotal")).alias("ImporteTotalCliente"))
)

ventas_por_cliente.show(5)
```

> [See performance \(1\)](#) [Optimize](#)

ventas_por_cliente: pyspark.sql.connect.dataframe.DataFrame = [IdCliente: integer, ImporteTotalCliente: decimal(31,2)]

IdCliente	ImporteTotalCliente
1	800.00
2	150.00
3	50.00

6.2. Modelo predictivo básico en Databricks ML

El modelo utiliza técnicas de Machine Learning (Regresión Lineal) para predecir el importe total que gastará cada cliente, basándose en su comportamiento histórico de compras.

A partir del conjunto de ventas, el sistema calcula automáticamente varias métricas por cliente, tales como:

- **Número de compras realizadas**
- **Cantidad total de productos adquiridos**
- **Variedad de productos distintos**
- **Precio medio pagado**
- **Importe medio por compra**
- **Recencia** (días desde la última compra)

Estas características se combinan en un vector de entrada para entrenar un modelo de **regresión lineal**, que aprende la relación entre estos patrones de compra y el gasto total del cliente.

Una vez entrenado, el modelo es capaz de **estimar el importe total esperado para nuevos clientes** o evaluar clientes existentes, generando una predicción muy precisa gracias a la fuerte correlación entre las variables de comportamiento y el gasto real.

Código del modelo:


```

# =====
# IMPORTS NECESARIOS
# =====
# Funciones de Spark SQL para crear agregaciones y columnas
from pyspark.sql.functions import (
    sum as _sum,
    avg,
    count,
    countDistinct,
    max,
    datediff,
    current_date
)

# Componentes para Machine Learning en Spark MLlib
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.regression import LinearRegression

# =====
# 1. CREACIÓN DEL DATASET DE FEATURES POR CLIENTE
# =====
# A partir del dataframe df_ventas creamos variables que describen el
# comportamiento de cada cliente. Estas variables servirán como entrada
# (features) para el modelo predictivo.

df_features = (
    df_ventas.groupBy("IdCliente")
    .agg(
        # Número de compras realizadas por el cliente
        count("*").alias("NumCompras"),

        # Número de productos diferentes que compró
        countDistinct("IdProducto").alias("NumProductosDistintos"),

        # Cantidad total de unidades compradas
        _sum("Cantidad").alias("CantidadTotal"),

        # Precio medio pagado por producto
        avg("PrecioUnitario").alias("PrecioMedio"),
    )
)

```

```

    # Importe medio por compra
    avg("ImporteTotal").alias("TicketMedio"),

    # Importe total gastado → será nuestro "label"
    _sum("ImporteTotal").alias("ImporteTotalCliente"),

    # Fecha de la última compra
    max("Fecha").alias("UltimaCompra")
)
)

# Añadimos una nueva feature:
# "RecenciaDias" = días desde la última compra hasta hoy.
# Muy usado en modelos de análisis RFM y predicción de gasto.
df_features = df_features.withColumn(
    "RecenciaDias",
    datediff(current_date(), df_features["UltimaCompra"])
)

# =====
# 2. PREPARACIÓN DE DATOS PARA ENTRENAR EL MODELO
# =====

# La columna a predecir (variable objetivo) se llamará "label".
df_ml = df_features.withColumnRenamed("ImporteTotalCliente", "label")

# Seleccionamos las columnas que serán las FEATURES del modelo.
input_cols = [
    "NumCompras",
    "NumProductosDistintos",
    "CantidadTotal",
    "PrecioMedio",
    "TicketMedio",
    "RecenciaDias"
]

# VectorAssembler convierte varias columnas numéricas en un único vector,
# que es el formato requerido por Spark ML.
assembler = VectorAssembler(inputCols=input_cols, outputCol="features")

```

```
# Aplicamos el assembler al dataset.
```

```
df_ml = assembler.transform(df_ml)
```

```
# =====
```

```
# 3. DIVISIÓN EN TRAIN / TEST
```

```
# =====
```

```
# Se divide el dataset en un conjunto de entrenamiento (80%)
```

```
# y un conjunto de test (20%) para evaluar el modelo.
```

```
train, test = df_ml.randomSplit([0.8, 0.2], seed=42)
```

```
# =====
```

```
# 4. ENTRENAMIENTO DEL MODELO (REGRESIÓN LINEAL)
```

```
# =====
```

```
# Creamos el modelo especificando qué columnas usar.
```

```
lr = LinearRegression(featuresCol="features", labelCol="label")
```

```
# Entrenamos el modelo con los datos de entrenamiento.
```

```
model = lr.fit(train)
```

```
# =====
```

```
# 5. GENERAR PREDICCIONES
```

```
# =====
```

```
# Aplicamos el modelo al conjunto de test.
```

```
preds = model.transform(test)
```

```
# Mostramos por cada cliente el valor real (label)
```

```
# vs. el valor predicho por el modelo (prediction).
```

```
preds.select("IdCliente", "label", "prediction").show()
```

IdCliente	label	prediction
3	119.00	119.99247484723418
7	58.00	57.99999828437586
9	128.00	127.99999856269221
14	158.00	157.99999865570874
20	242.50	242.4999986673132
24	218.00	217.99999884174179
30	317.50	317.49999868367706
36	72.50	72.50000017943944
46	87.50	87.50000087448029
47	178.00	178.0000003858621
48	271.50	271.499998633101
50	467.50	467.49999871640455
52	193.00	193.00000064854794
56	102.50	102.50000156952115
63	339.00	339.0000003968636
70	617.50	617.499998749132
78	406.50	406.50000093041683

6.3. Registro del experimento y del modelo en AI/ML (Experiments y Models)

Para integrar el trabajo con la sección AI/ML de Databricks, el entrenamiento se registra con MLflow. Esto genera automáticamente un experimento visible en el apartado “Experiments” y un modelo registrado en “Models”, ambos asociados al lakehouse. Es necesario crear un volumen en nuestro catalog, por lo que volvemos al que teníamos y en el schema que hemos creado antes le damos a create y a volume

Create a new volume

Volumes are locations for storing files in Unity Catalog.

Volume name*

volumen_ml

Volume type [Learn more](#)

☒ Managed volume
Files are stored in a location managed by Unity Catalog.
 ☐ External volume
Files are stored in an external location that you configure.

Choose catalog and schema*

Volumes are organized under a catalog and a schema. Select existing or create new.

azdb_proyecto_mario

lakeouse_proyecto

Cancel

Create

Ahora con el volumen creado podemos registrar el modelo predictivo en databricks AI/ML

Código de registro:

```
import mlflow
from mlflow.models import infer_signature

mlflow.set_registry_uri("databricks-uc")

registered_name =
"azdb_proyecto_mario.lakeouse_proyecto.modelo_ventas_clientes"

signature = infer_signature(
    train.select("features").toPandas(),
    preds.select("prediction").toPandas()
)

# Replace with your actual Unity Catalog volume path
dfs_tmpdir =
"/Volumes/azdb_proyecto_mario/lakeouse_proyecto/volumen_ml/mlflow_tmp"

with mlflow.start_run(run_name="demo_ventas_por_cliente"):
    mlflow.log_param("algorithm", "LinearRegression")
    rmse = model.summary.rootMeanSquaredError
    mlflow.log_metric("rmse", rmse)
    mlflow.spark.log_model(
        model,
        artifact_path="model",
        registered_model_name=registered_name,
        signature=signature,
        dfs_tmpdir=dfs_tmpdir
    )
```

```
import mlflow
from mlflow.models import infer_signature

mlflow.set_registry_uri("databricks-uc")

registered_name = "azdb_proyecto_mario.lakehouse_proyecto.modelo_ventas_clientes"

signature = infer_signature(
    train.select("features").toPandas(),
    preds.select("prediction").toPandas()
)

# Replace with your actual Unity Catalog volume path
dfs_tmpdir = "/Volumes/azdb_proyecto_mario/lakehouse_proyecto/volumen_ml/mlflow_tmp"

with mlflow.start_run(run_name="demo_ventas_por_cliente"):
    mlflow.log_param("algorithm", "LinearRegression")
    rmse = model.summary.rootMeanSquaredError
    mlflow.log_metric("rmse", rmse)
    mlflow.spark.log_model(
        model,
        artifact_path="model",
        registered_model_name=registered_name,
        signature=signature,
        dfs_tmpdir=dfs_tmpdir
    )

> See performance (3)
```

▼ (1) MLflow run
Logged 1 run to an experiment in MLflow. Learn more

2025/12/03 20:12:23 WARNING mlflow.models.signature: Failed to infer schema for inputs. Setting schema to 'Schema([ColSpec(type=AnyType())])' as default. Note that MLflow does not validate data types during inference for AnyType. To see the full traceback, set logging level to DEBUG.
2025/12/03 20:12:28 WARNING mlflow.utils.requirements_utils: Found pyspark version (4.0.0+databricks.connect.17.2.2) contains a local version label (+databricks.connect.17.2.2). MLflow logged a pip requirement for this package as 'pyspark==4.0.0' without the local version label to make it installable from PyPI. To specify pip requirements containing local version labels, please use 'conda_env' or 'pip_requirements'.
2025/12/03 20:12:32 WARNING mlflow.utils.environment: Encountered an unexpected error while inferring pip requirements (model URI: /local_disk0/user_tmp_data/spark-81e11a29-7f5c-48a1-9d3b-c7/tmpk6nirf/model, flavor: spark). Fall back to return ['pyspark==4.0.0']. Set logging level to DEBUG to see the full traceback.
Successfully registered model 'azdb_proyecto_mario.lakehouse_proyecto.modelo_ventas_clientes'.
Created version '1' of model 'azdb_proyecto_mario.lakehouse_proyecto.modelo_ventas_clientes'.

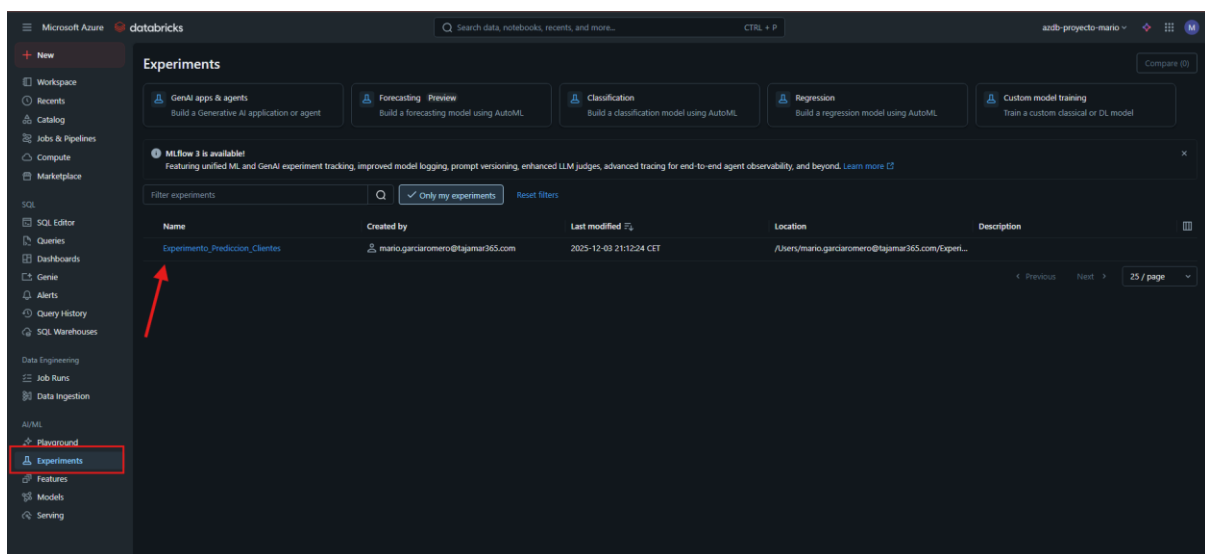
Resultado tras ejecutar la celda

1. AI/ML → Experiments

Aparece un nuevo experimento llamado **demo_ventas_por_cliente**, que contiene:

- El parámetro registrado (algorithm = LinearRegression)
- La métrica de error **RMSE**
- El artefacto del modelo entrenado
- El historial completo de la ejecución

Esto permite auditar el entrenamiento y compararlo con futuras versiones.



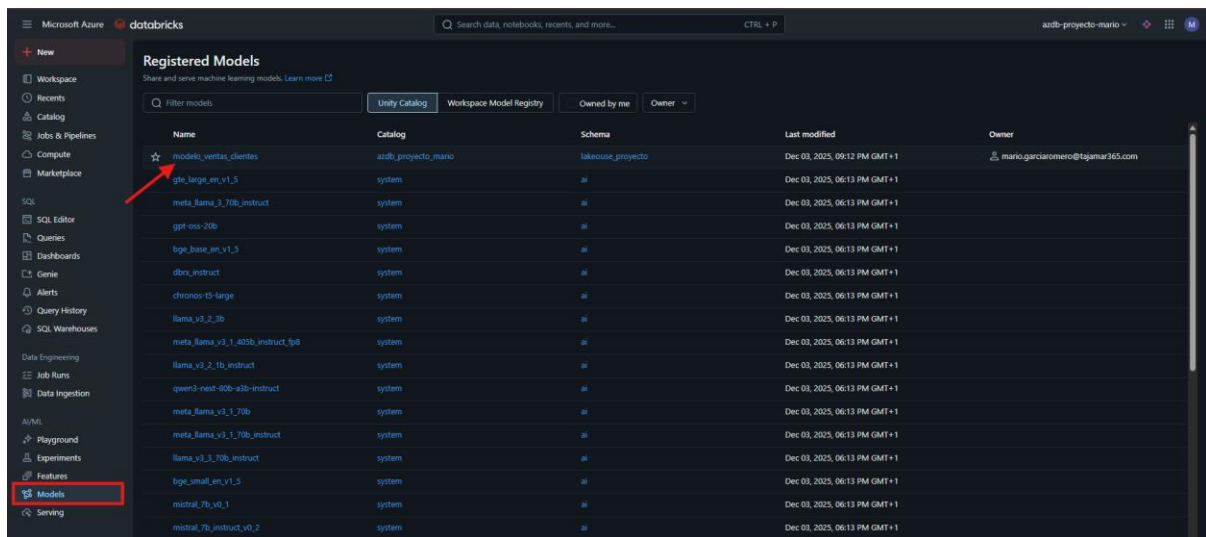
2. AI/ML → Models

En el apartado **Models**, dentro del catálogo y esquema del proyecto, aparece el modelo:

azdb_proyecto_mario.lakehouse_proyecto.modelo_ventas_clientes

Este modelo queda registrado en Unity Catalog, con versión 1, y puede:

- Ser descargado
- Volverse a cargar para inferencia
- Promocionarse a producción
- Monitorizarse desde Databricks



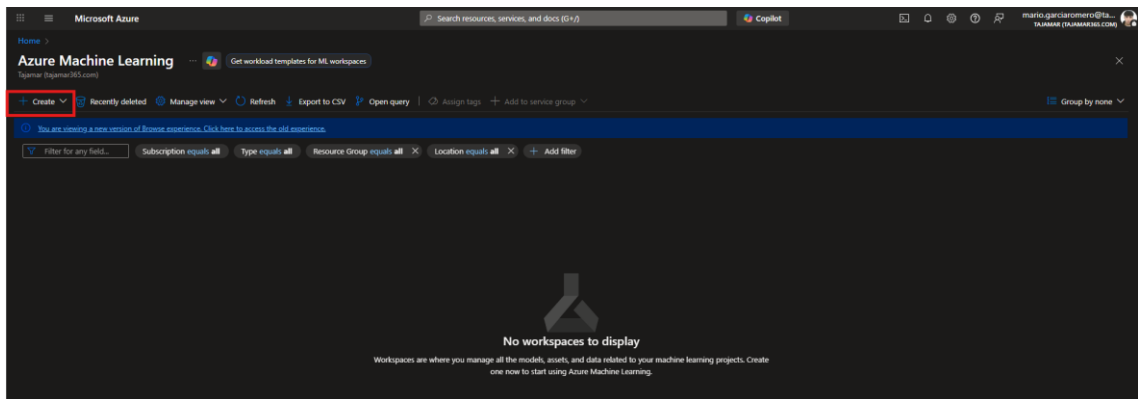
Name	Catalog	Schema	Last modified	Owner
☆ modelo_ventas_clientes	azdb_proyecto_mario	lakehouse_proyecto	Dec 03, 2025, 09:12 PM GMT + 1	mario.garciaomero@tjamar365.com
gpt_large_en_v1_5	system	ai	Dec 03, 2025, 06:13 PM GMT + 1	
meta_llama_v3_70b_instruct	system	ai	Dec 03, 2025, 06:13 PM GMT + 1	
gpt-oss-20b	system	ai	Dec 03, 2025, 06:13 PM GMT + 1	
llgpt_base_en_v1_5	system	ai	Dec 03, 2025, 06:13 PM GMT + 1	
cllm_instruct	system	ai	Dec 03, 2025, 06:13 PM GMT + 1	
chronos-t5-large	system	ai	Dec 03, 2025, 06:13 PM GMT + 1	
llama_v3_2_3b	system	ai	Dec 03, 2025, 06:13 PM GMT + 1	
meta_llama_v3_1_405b_instruct_5ps	system	ai	Dec 03, 2025, 06:13 PM GMT + 1	
llama_v3_2_1b_instruct	system	ai	Dec 03, 2025, 06:13 PM GMT + 1	
qwen3-next-80b-a3b-instruct	system	ai	Dec 03, 2025, 06:13 PM GMT + 1	
meta_llama_v3_1_70b	system	ai	Dec 03, 2025, 06:13 PM GMT + 1	
meta_llama_v3_1_70b_instruct	system	ai	Dec 03, 2025, 06:13 PM GMT + 1	
llama_v3_2_70b_instruct	system	ai	Dec 03, 2025, 06:13 PM GMT + 1	
llgpt_small_en_v1_5	system	ai	Dec 03, 2025, 06:13 PM GMT + 1	
mistral_7b_v0_1	system	ai	Dec 03, 2025, 06:13 PM GMT + 1	
mistral_7b_instruct_v0_2	system	ai	Dec 03, 2025, 06:13 PM GMT + 1	

Para tu proyecto basta con que expliques cómo **Azure ML accede a los mismos datos físicos del lakehouse**, es decir, al contenedor ADLS donde están tus tablas Delta. No hace falta que lea “la tabla Delta” como tal, sino la carpeta/parquet de detrás.[learn.microsoft+1](https://learn.microsoft.com/en-us/azure/ml/conceptual/how-to-prepare-data-for-training)

7. Conexión con azure ml

7.1. Crear el workspace de Azure ML

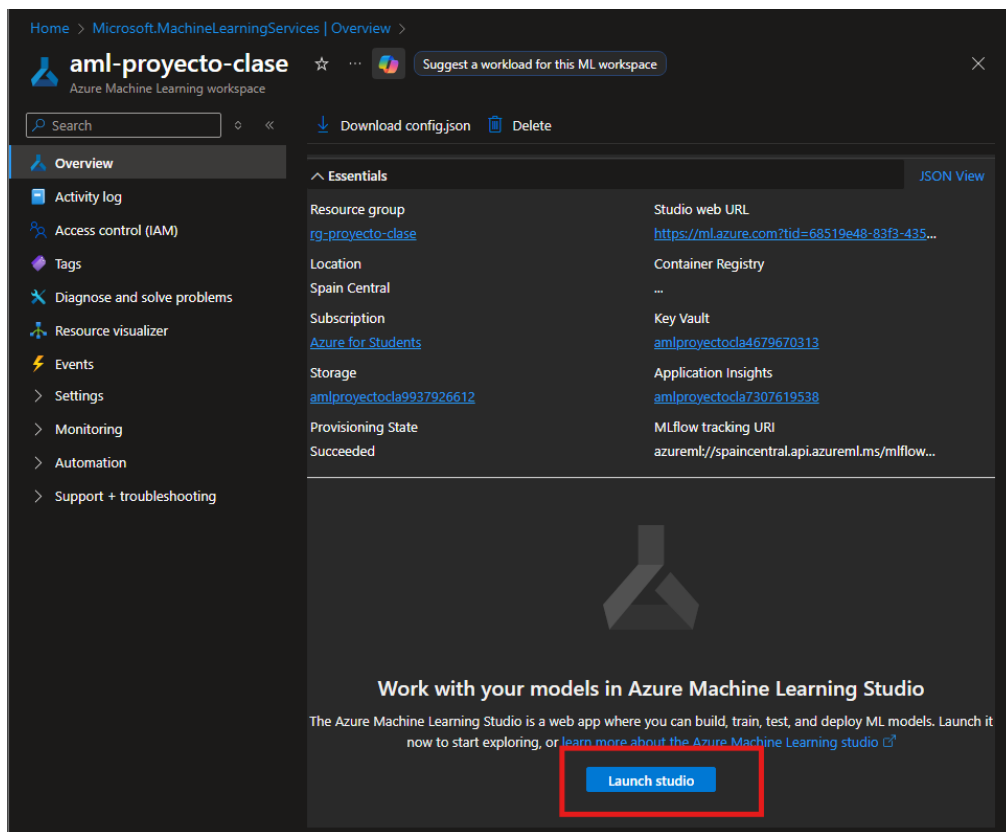
1. Entra en el portal de Azure (<https://portal.azure.com>).
2. Pulsa “Crear un recurso” y busca “Azure Machine Learning”.
3. Selecciona “Azure Machine Learning” → “Crear” → new workspace.



4. Rellena: suscripción, grupo de recursos, nombre del workspace (por ejemplo, aml-proyecto-clase), región y deja el resto por defecto.

5. Pulsa “Revisar y crear” y espera a que se despliegue el recurso.

Cuando termine, abre el recurso y haz clic en Launch studio para entrar en Azure ML Studio.

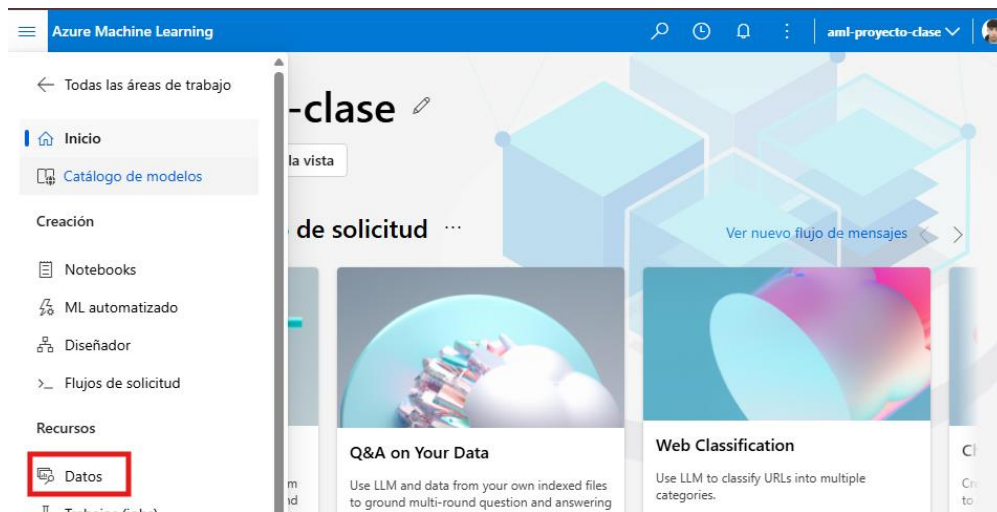


7.2. Crear un datastore a tu Data Lake

En primer lugar, se configura en Azure Machine Learning un datastore que apunta al mismo almacenamiento que utiliza el lakehouse de Databricks.

Pasos en Azure ML Studio:

1. Entra en **Azure Machine Learning Studio** (tu workspace).
2. En el menú izquierdo, ve a **Datos** → **Almacenes de datos**.



3. Pulsa + Create.

Datos

Recursos de datos **Almacenes de datos** Importación de datos VISTA PREVIA Conexiones de datos VISTA PREVIA

Los almacenes de datos se conectan de forma segura a un servicio de almacenamiento en Azure mediante el almacenamiento de información de conexión. Con los almacenes de datos, ya no necesita proporcionar la información de credenciales en los scripts para acceder a los datos. [Más información sobre los almacenes de datos](#)

Los almacenes de datos de OneLake usados para conectarse a los datos del punto de conexión de Microsoft Fabric solo se pueden crear a través de la CLI y el SDK. [Más información sobre el almacén de datos de OneLake](#)

+ Crear Actualizar Cancelar registro Establecer como almacén de datos predeterminado Restablecer vista

Buscar Filtrar Columnas

Nombre	Tipo	Nombre de almacenamiento
workspaceworkingdirectory	Recurso compartido de archivos de Azure	amlproyectocla9937926612
workspacefilestore	Recurso compartido de archivos de Azure	amlproyectocla9937926612
workspaceartifactstore	Azure Blob Storage	amlproyectocla9937926612
workspaceblobstore (Predeter...	Azure Blob Storage	amlproyectocla9937926612

4. Tipo de almacenamiento: **Azure Data Lake Storage Gen2** (o Azure Blob si tu contenedor está así publicado).

5. Rellena:

- Suscripción.
- Storage account: la misma que usa el lakehouse (por ejemplo stproyectoclase).
- File system / Container: el contenedor donde tienes los datos (deltalake o el que uses).
- Método de autenticación: **Account**, y ponemos la que hemos tenido que poner anteriormente, key1.

Crear almacén de datos

Nombre del almacén de datos *

almacen_proyecto

Tipo de almacén de datos *

Azure Blob Storage

Método de selección de cuentas

☒ De suscripción de Azure

☐ Especificar manualmente

Id. de suscripción *

Azure for Students (a8bdc78-254e-4a67-bd98-a1f016ee5615)

Cuenta de almacenamiento *

stproyectoclase (rg-proyecto-clase)

Contenedor de blobs *

deltalake

☒ Guardar credenciales con el almacén de datos para acceder a ellos ⓘ

Tipo de autenticación * ⓘ

Clave de cuenta

Clave de cuenta * ⓘ

.....

☒ Usar la identidad administrada del área de trabajo para obtener una vista previa y generar perfiles de los datos en Azure Machine Learning Studio ⓘ

ⓘ Nota: Azure Machine Learning Service no valida si el origen de datos subyacente existe o ...

Crear Cancelar

6. Dale un nombre al datastore, por ejemplo, almacen_proyecto, y crea.

Con esto, Azure ML ya “ve” tu Data Lake.

7.3. Crear un Data asset apuntando a la ruta del lakehouse

A continuación se registra un Data asset que apunta a la carpeta concreta del Data Lake que contiene los datos del proyecto (por ejemplo la capa bronze o las salidas Delta del ETL).

1. En Azure ML Studio, sigue en **Data**, pestaña **recursos de datos**.
2. Rellenamos la primera pantalla, nombre: lakehouse_asset_ventas, descripcion(opcional), tipo Tabular y le damos a siguiente

Crear recurso de datos

- 1 Tipo de datos
- 2 Origen de datos

Establecer el nombre y el tipo del recurso de datos

Nombre *

lakehouse_asset_ventas

Descripción

Datos de ventas procedentes del lakehouse de Databricks (ADLS Gen2).

Tipo * ①

Tabular

Tipos de recursos de datos (de las API de Azure ML v2)

Archivo (uri_file)

Carpeta (uri_folder)

Tabla (mtable)

Tipos de conjunto de datos (de las API de Azure ML v1)

Tabular

Archivo

3. Pulsa + Create → Desde Azure Storage.

Crear recurso de datos

- ✓ Tipo de datos
- Origen de datos

Elegir un origen para el recurso de datos

Elija el origen de datos desde el que desea crear el recurso. Un origen de datos puede ser de una ubicación de almacenamiento local del equipo, de un almacén de datos adjunto, de Azure Storage o de una ubicación web disponible públicamente.

Desde Azure Storage

Cree un recurso de datos a partir de servicios de almacenamiento de datos registrados, incluidos Azure Blob Storage, el recurso compartido de archivos de Azure y Azure Data Lake.

De archivos locales

Cree un recurso de datos cargando archivos desde la unidad local.

Desde bases de datos SQL

Cree un conjunto de datos a partir de una base de datos de Azure SQL y una base de datos de Azure PostgreSQL.

De archivos web

Cree un recurso de datos a partir de un único archivo ubicado en una dirección URL web pública.

Desde Azure Open Datasets

Cree un conjunto de datos con un solo clic a partir de conjuntos de datos predefinidos. Estos conjuntos de datos los crea el público general y se publican como Azure Open Datasets.

4. Elige el datastore almacen_proyecto.

3

Tipo de almacenamiento de origen

4

Ruta de acceso de almacenamiento

5

Configuración

6

Esquema

7

Revisar

Tipo de almacén de datos *

Azure Blob Storage

Crear nuevo almacén de datos

Buscar almacén de datos

Filtrar

Columnas

Nombre ↓	Nombre de almacenamiento
workspaceblobstore	amlproyectocla9937926612
workspaceartifactstore	amlproyectocla9937926612
almacen_proyecto	stproyectoclase

5. En el campo **Path**, escribe la ruta dentro del contenedor, por ejemplo:

a. bronze/ventas/

Crear recurso de datos

- Tipo de datos
- Origen de datos
- Tipo de almacenamiento de origen
- Ruta de acceso de almacenamiento**
- Configuración
- Esquema
- Revisar

Elegir una ruta de acceso de almacenamiento

Navegue hasta o escriba la ruta de acceso al almacenamiento que desea usar para este recurso de datos.

☒ Ir a la ruta de acceso de almacenamiento

☐ Escribir la ruta de acceso de almacenamiento manualmente

Ruta seleccionada: bronze/ventas/**

Actualizar contenido del almacén de datos

Navegar al nivel superior

Filtrar...

	Nombre	Fecha de cr
☐	clientes/	--
☐	productos/	--
☒	ventas/	--

6. Revisamos el archivo

Crear recurso de datos

- ✓ Tipo de datos
- ✓ Origen de datos
- ✓ Tipo de almacenamiento de origen
- ✓ Ruta de acceso de almacenamiento
- 5 Configuración**
- 6 Esquema
- 7 Revisar

Configuración

Esta configuración determina cómo se analizan los datos. La configuración inicial se detecta automáticamente; puede cambiarlos según sea necesario para reanalizar los datos.

Formato de archivo

Archivo de Parquet

Vista previa de datos

IdVenta	Fecha	IdCliente	IdProducto	Cantidad	PrecioUn...	Importe...
1	2025-01-...	1	1	1	800	800
2	2025-01-...	2	3	1	150	150
3	2025-01-...	3	2	2	25	50
4	2025-01-...	2	2	2	21.5	43
5	2025-01-...	3	3	3	23	69
6	2025-01-...	4	4	4	24.5	98
7	2025-01-...	5	5	5	26	130
8	2025-01-...	6	6	1	27.5	27.5
9	2025-01-...	7	7	2	29	58
10	2025-01-...	8	8	3	30.5	91.5
11	2025-01-...	9	9	4	32	128
12	2025-01-...	10	10	5	33.5	167.5
13	2025-01-...	11	11	1	35	35
14	2025-01-...	12	12	2	36.5	73
15	2025-01-...	13	13	3	38	114
16	2025-01-...	14	14	4	39.5	158

Atrás

Siguiente

Revisar

Cancelar

7. Y lo creamos

Crear recurso de datos

- ✓ Tipo de datos
- ✓ Origen de datos
- ✓ Tipo de almacenamiento de origen
- ✓ Ruta de acceso de almacenamiento
- ✓ Configuración
- ✓ Esquema
- 8 Revisar**

Ruta de acceso de almacenamiento

Ruta de acceso del archivo seleccionado: `bronze/ventas/**`

Configuración

Delimitador: Custom

Codificación: UTF-8

Formato de archivo: Archivo de Parquet

Encabezados de columna: Todos los archivos tienen los mismos encabezados

Número de filas que se omitirán: None

El conjunto de datos contiene datos de varias líneas: false

Atrás

Crear

Cancelar

Ese Data asset es la “puerta de entrada” de Azure ML a los mismos ficheros que usa Databricks.

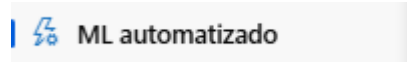
7.4. Usar el Lakehouse en un experimento de Azure ML

Una vez creado el Data asset, se puede utilizar directamente como dataset de entrada en experimentos y notebooks de Azure ML.

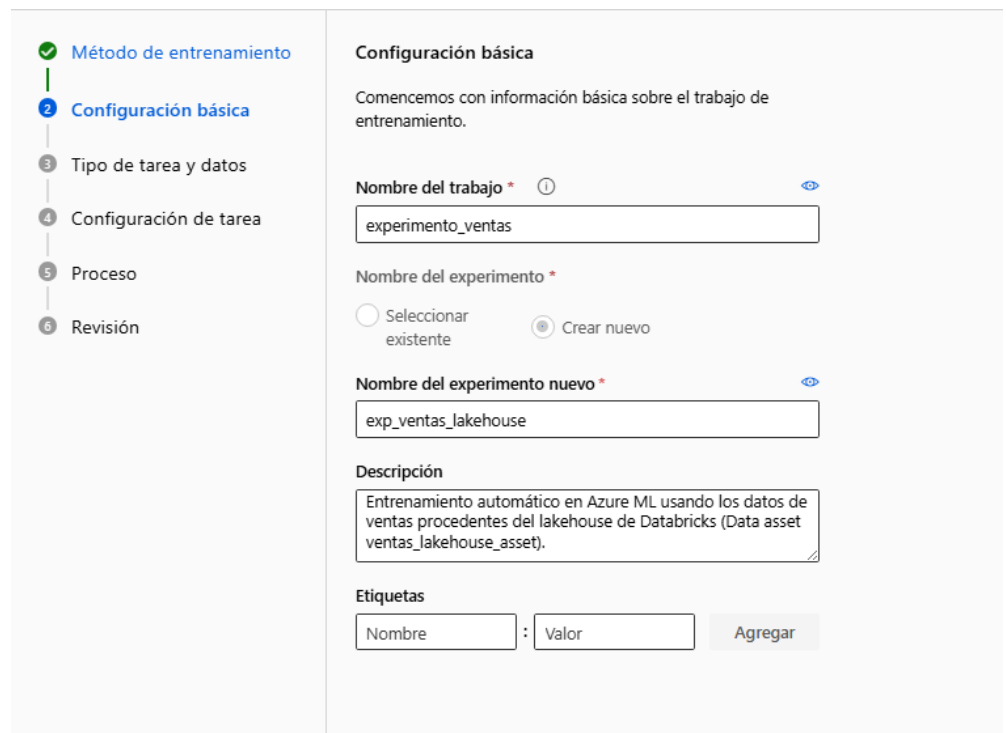
Dos usos típicos:

- **Automated ML:**

- En Azure ML Studio, ve a **Automated ML**.



- Crea un nuevo experimento y
 - Nombre del trabajo: por ejemplo experimento_ventas_lakehouse
 - Nombre del experimento: deja marcada la opción Crear nuevo y en Nombre del experimento nuevo pon: exp_ventas_lakehouse
 - Descripción (opcional, pero ayuda en la memoria): Entrenamiento automático en Azure ML usando los datos de ventas procedentes del lakehouse de Databricks (Data asset ventas_lakehouse_asset).

A screenshot of the 'Configuración básica' (Basic Configuration) form in the Azure ML Studio 'Automated ML' interface. The form is part of a multi-step process, with 'Configuración básica' being the second step. The left sidebar shows a progress bar with steps: 1. Método de entrenamiento (completed), 2. Configuración básica (current), 3. Tipo de tarea y datos, 4. Configuración de tarea, 5. Proceso, and 6. Revisión. The main form area contains the following fields: 'Nombre del trabajo *' with the value 'experimento_ventas'; 'Nombre del experimento *' with radio buttons for 'Seleccionar existente' (unselected) and 'Crear nuevo' (selected); 'Nombre del experimento nuevo *' with the value 'exp_ventas_lakehouse'; 'Descripción' with the text 'Entrenamiento automático en Azure ML usando los datos de ventas procedentes del lakehouse de Databricks (Data asset ventas_lakehouse_asset)'; and 'Etiquetas' with a field for 'Nombre' containing 'Valor' and an 'Agregar' button.

- Tipo de tarea regresión y seleccionamos el dataset

✓ Método de entrenamiento

✓ Configuración básica

3 Tipo de tarea y datos

4 Configuración de tarea

5 Proceso

6 Revisión

Tipo de tarea y datos

Elija el tipo de tarea que desea que realice el modelo y los datos que se van a usar para el entrenamiento. [Más información](#)

Seleccionar el tipo de tarea *

Regresión

Seleccionar datos

Asegúrese de que los datos están preprocesados en un formato compatible.

+ Crear

Actualizar

...

Buscar

Filtrar

Columnas

Nombre	Tipo	Fecha de cr...
lakehouse_asset_ventas	Tabla	Dec 3, 2025 9:...

<<

<

Página 1 de 1

>

>>

25/Página

Atrás

Siguiente

Cancelar

- Indicas la columna objetivo, la que se va a querer predecir (Cantidad)

Configuración de tarea

Tipo de tarea

Regresión

Datos

lakehouse_asset_ventas [\(Ver datos\)](#)

Columna de destino *

Cantidad (Integer)

Ver opciones de configuración adicionales

Ver configuración de caracterización

> Límites

Validar y probar

Puede elegir un tipo de validación y seleccionar datos de prueba como un paso opcional.

Tipo de validación

Automático

Datos de prueba

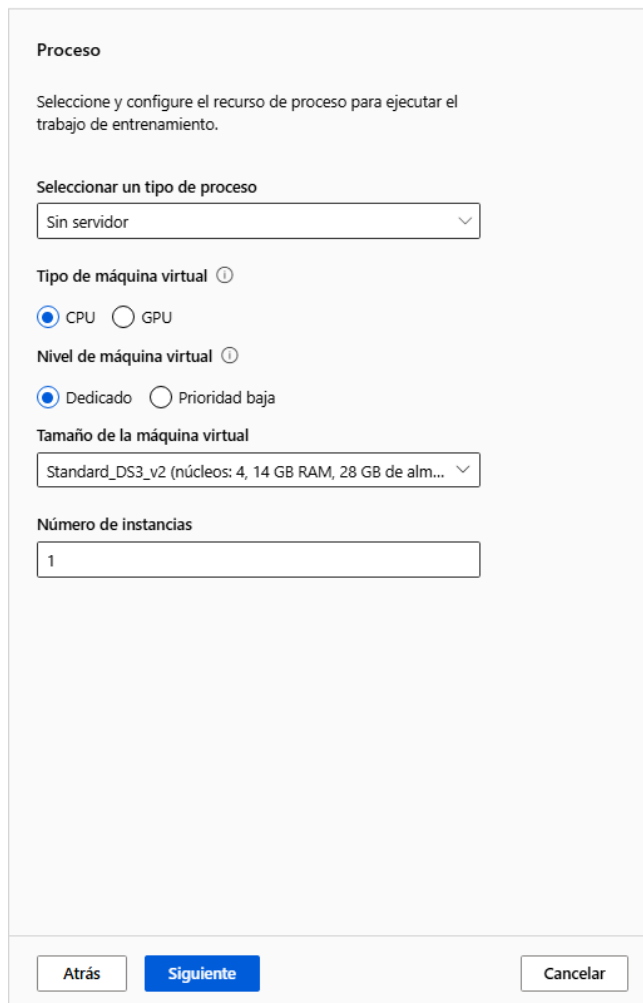
Ninguno

Atrás

Siguiente

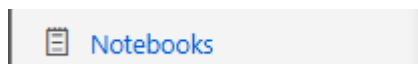
Cancelar

- En el proceso dejamos todo por defecto

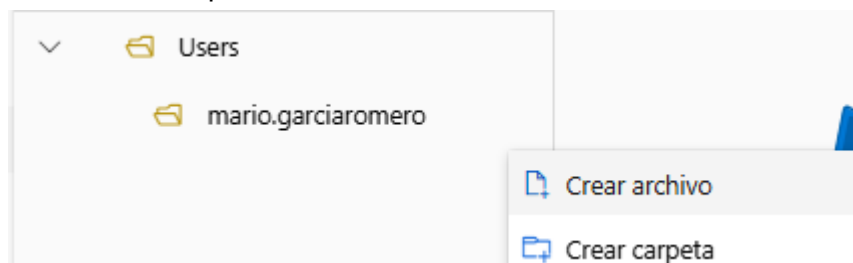


- **Notebook en Azure ML:**

- Ve a **Notebooks** en Azure ML Studio, crea uno en Python.



- En nuestra carpeta le damos a crear archivo



- Aquí creamos el notebook con el nombre que queramos y ya podríamos ir probando cosas

Crear archivo

Ubicación de archivo

Users/mario.garciaaromero  Editar ubicación

Nombre de archivo *

lakehouse_ventas.ipynb

Tipo de archivo

Cuaderno (*.ipynb) 

☐ Sobrescribir si ya existe

Crear

Cancelar

Azure Machine Learning se conecta al mismo Lakehouse que Databricks registrando un datastore contra el contenedor de ADLS Gen2 y creando un Data asset (por ejemplo ventas_lakehouse_asset) que apunta a la ruta de datos utilizada por Databricks. Este Data asset se puede seleccionar como dataset de entrada en experimentos de Azure ML (Automated ML o notebooks), garantizando que ambos entornos trabajan sobre las mismas fuentes de datos.

En este proyecto se ha integrado Azure Machine Learning conectándolo directamente a la cuenta de almacenamiento (Azure Data Lake Storage Gen2) donde Databricks guarda las tablas Delta del lakehouse. De esta forma, Azure ML trabaja sobre los mismos ficheros Parquet/Delta que utiliza Databricks, a través de un Data asset configurado sobre el Storage Account. No obstante, esta no es la única opción posible: también sería viable conectar Azure ML directamente a Databricks usando el endpoint SQL (SQL Warehouse) y una conexión JDBC/ODBC autenticada con tokens, de modo que Azure ML pudiera leer las tablas Delta como tablas SQL. En esta práctica se ha elegido la primera alternativa, por ser más sencilla de implementar y suficiente para demostrar que Databricks y Azure ML comparten el mismo lakehouse de datos.